Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Факультет международных образовательных программ»

КАФЕДРА «Компьютерные системы автоматизации производства»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ

НА ТЕМУ:

«Система распознавания движущихся производственных объектов в облачной системе управления»

Студент РК9И-85Б _____ Т.Ш. Фам
(Подпись, дата)

Руководитель ВКР _____ С.А. Буханов
(Подпись, дата)

Нормоконтролер _____ С.И. Гавриленков
(Подпись, дата)

2022 г.

РЕФЕРАТ

Выпускная квалификационная работа выполнена по теме: «Система распознавания движущихся производственных объектов в облачной системе управления».

Расчетно-пояснительная записка содержит 82 листа формата А4, 25 рисунков, 11 листингов, 17 используемых источников, 5 приложений, 3 таблицы.

Целью выполнения работы является создание системы распознавания движущихся производственных объектов в облачной системе управления, входящей в состав учебной ГПС Denford. В работе проведен анализ существующего решения, выявлены его недостатки. Разработаны алгоритмы обнаружения и идентификации движущихся производственных объектов на основе библиотеки OpenCV, результаты которых хранятся на облачной платформе Dropbox. Осуществлен контроль системы распознавания объектов веб-приложением по сети Интернет через протокол MQTT от облачной платформы Yandex Cloud. Программная часть системы была реализована на языке Python, а аппаратная располагается на одноплатном компьютере Raspberry Pi 3 Model B.

В заключении описаны результаты проделанной работы.

Ключевые слова: ГПС. обнаружение. идентификация, Python, Opencv, облачная платформа, Dropbox, Yandex Cloud, IIoT. IoT.

СОДЕРЖАНИЕ

РЕФЕРАТ 2
СПИСОК ОБОЗНАЧЕНИЙ И СОКРАЩЕНИЙ 5
ВВЕДЕНИЕ 7
1. Предпроектное исследование 9
1.1 Объект и предмет исследования 9
1.2 Анализ принципов работы существующей системы 11
1.3 Недостатки существующей системы 13
1.4 Одноплатный компьютер 14
2. Концептуальное проектирование системы 21
2.1 Постановка задачи 21
2.2 Разработка функциональной диаграммы системы распознавания 22
2.3 Разработка архитектуры системы распознавания объектов в облачной системе управления 25
3. Техническое задание 29
3.1 Общие сведения 29
3.2 Назначение и цели создания системы 29
3.3 Требования к системе в целом 30
4. Структурное проектирование 32
4.1 Сравнительный анализ и выбор одноплатного компьютера 32
4.2 Взаимодействие Raspberry Pi 3 с веб-приложением через облачную платформу Yandex.Cloud 36
4.3 Формат сообщений между облачной платформой Yandex.Cloud и Raspberry Pi 3 39
4.4 Взаимодействие Raspberry Pi 3 с модулем камеры 40
4.5 Взаимодействие Raspberry Pi 3 с файловым хостингом Dropbox 43
4.6 Сравнительный анализ и выбор алгоритма обнаружения движения объектов 45
4.7 Сравнительный анализ и выбор алгоритма поиска регионов 48
4.8 Сравнительный анализ и выбор алгоритма распознавания объекта 50

5. Рабочее проектирование 52
5.1 Реализация подключения Raspberry Pi 3 к камере 55
5.2 Реализация подключения Raspberry Pi 3 к файловому хостингу Dropbox. 56
5.3 Реализация основных алгоритмов системы распознавания движущихся производственных объектов 57
6. Апробирование 65
ЗАКЛЮЧЕНИЕ 67
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 68
Приложение А 70
Приложение Б 72
Приложение В 74
Приложение Г 75
Приложение Д 76

СПИСОК ОБОЗНАЧЕНИЙ И СОКРАЩЕНИЙ

IoT (Internet of Things) – система взаимосвязанных устройств (вещей), оснащенных средствами для беспроводной передачи данных между друг другом, внешней средой без непосредственного вмешательства человека.

IIoT (Industrial Internet of Things) – взаимосвязанные датчики, исполнительные устройства, промышленное оборудование и другие девайсы, объединенные в сеть совместно с промышленным программным обеспечением для управления производством.

ГПС (Гибкая производственная система) – совокупность в разных сочетаниях оборудования с ЧПУ, роботизированных технологических комплексов, гибких производственных модулей, отдельных единиц технологического оборудования и систем обеспечения их функционирования в автоматическом режиме в течение заданного интервала времени, обладающую свойством автоматизированной переналадки при производстве изделий произвольной номенклатуры в установленных пределах значений их характеристик.

Python – высокоуровневый язык программирования общего назначения с динамической строгой типизацией и автоматическим управлением памятью, ориентированный на повышение производительности разработчика, читаемости кода и его качества, а также на обеспечение переносимости написанных на нём программ.

OpenCV (Open Source Computer Vision Library) – библиотека алгоритмов компьютерного зрения, обработки изображений и численных алгоритмов общего назначения с открытым кодом.

Raspberry Pi 3 model B – полноценный одноплатный компьютер размером с банковскую карту.

Yandex Cloud – облачная платформа от российской компании Yandex, используемая в данном курсовом проекте.

MQTT (Message Queuing Telemetry Transport) – упрощённый сетевой протокол, работающий поверх TCP/IP, ориентированный на обмен сообщений между устройствами по принципу издатель-подписчик.

MQTT-брокер – сервер-посредник, используемый в протоколе MQTT.

Dropbox – файловый хостинг, включающий персональное облачное хранение, синхронизацию файлов и клиент-программу.

Dropbox Core API v2 – расширение Dropbox, которое обеспечивает разработчикам доступ к таким функциям Dropbox, как загрузка, совместное использование, поиск и восстановление файлов, непосредственно в своих приложениях и работает на основе протокола HTTP.

ВВЕДЕНИЕ

Интернет вещи (Internet of Things или сокращенно IoT) занимает одно из важнейших мест в современной автоматизации. IoT представляет собой совокупность физических объектов, подключенных к интернету, образуя компьютерную сеть, и обменивающихся данными в реальном времени без участия человека. Позже появился промышленный интернет вещей (Industrial Internet of Things или сокращенно IIoT).

Реализацией концепции IIoT будет система взаимодействия отдельных устройств, систем и пользователей в рамках поддержки жизненного цикла продукции на основе открытых сетей. Для того чтобы создать умное производство, необходимо также применять технологии интернета вещей, которые позволяют в автоматическом режиме вести мониторинг состояния оборудования, контролировать производственный процесс не только в рамках одного производственного модуля, но и в рамках всего предприятия в целом [1]. Эти задачи решает система технического зрения. Её обычно разделяют на 2 подсистемы:

- Обнаружения объекта;
- Контроль и анализ объекта.

В свою очередь, каждая из этих подсистем включает в себя различный набор компонентов в зависимости от поставленной задачи.

Принцип работы системы технического зрения заключается в том, что она обнаруживает неподвижный или движущийся объект, захватывает изображение этого объекта, проверяет определенные свойства этого изображения, а затем передает результаты различных этапов контроля. Таким образом, различные этапы процесса визуального контроля включают:

- Обнаружение объекта и запуск захвата изображения в соответствующий момент;

- Захват подходящего изображения объекта (требуются надлежащее освещение, правильное фокусное расстояние и специальные светофильтры);
- Выполнение этапов контроля по захваченному изображению такие, как идентификация изделия, определение неправильной формы или недостающей части объекта, определение местоположение и др.;
- Вывод результатов этапов контроля.

Система технического зрения имеет множество преимуществ по сравнению человеком:

- Безостановочный контроль;
- Доступен сразу после настройки системы;
- После установки дополнительные расходы отсутствуют;
- Зависит от характеристик системы. Может быть легко изменена в соответствии с требованиями;
- Глобальный результат контроля + дополнительные сигналы результатов могут быть считаны любой системой и автоматически обработаны.

Облачные вычисления являются одной из важнейших технологий для реализации концепции IIoT. Облачные вычисления — это технология, предоставляющая вычислительных ресурсов, включая хранилище, вычислительные мощности, базы данных, сети, аналитику, искусственный интеллект и программные приложения, через Интернет. Интеграция системы технического зрения, в частности, системы обнаружения движения с облачными вычислениями даёт такие преимущества, как уменьшение затрат, увеличение быстродействия работы, и др.

1. Предпроектное исследование

1.1 Объект и предмет исследования

Объектом исследования в настоящей работе является ГПС Denford.

Предметом исследования в настоящей работе является система технического зрения, входящей в состав гибкого производственного модуля. Она расположена в лабораторной аудитории 297 кафедры «Компьютерные системы автоматизации производства» МГТУ им. Н. Э. Баумана. Система контроля является важнейшим модулем гибкой производственной системы, так как именно она определяет возможность безлюдного производственного процесса. ГОСТ 16504-81 [2] определяет задачи технического контроля как проверку соответствия контролируемого объекта установленным техническим требованиям. При этом информация о фактическом состоянии объекта называется первичной; обнаружение несоответствия (расхождения) между фактическими и требуемыми данными названо получением вторичной информации. В систему контроля ГПС Denford входит система технического зрения, которая идентифицирует производственный объект. следит за качеством изделия и передает информацию об объекте и рассогласованиях с моделями производственных процессов для принятия решения на различных уровнях управления ГПС (рисунок 1).

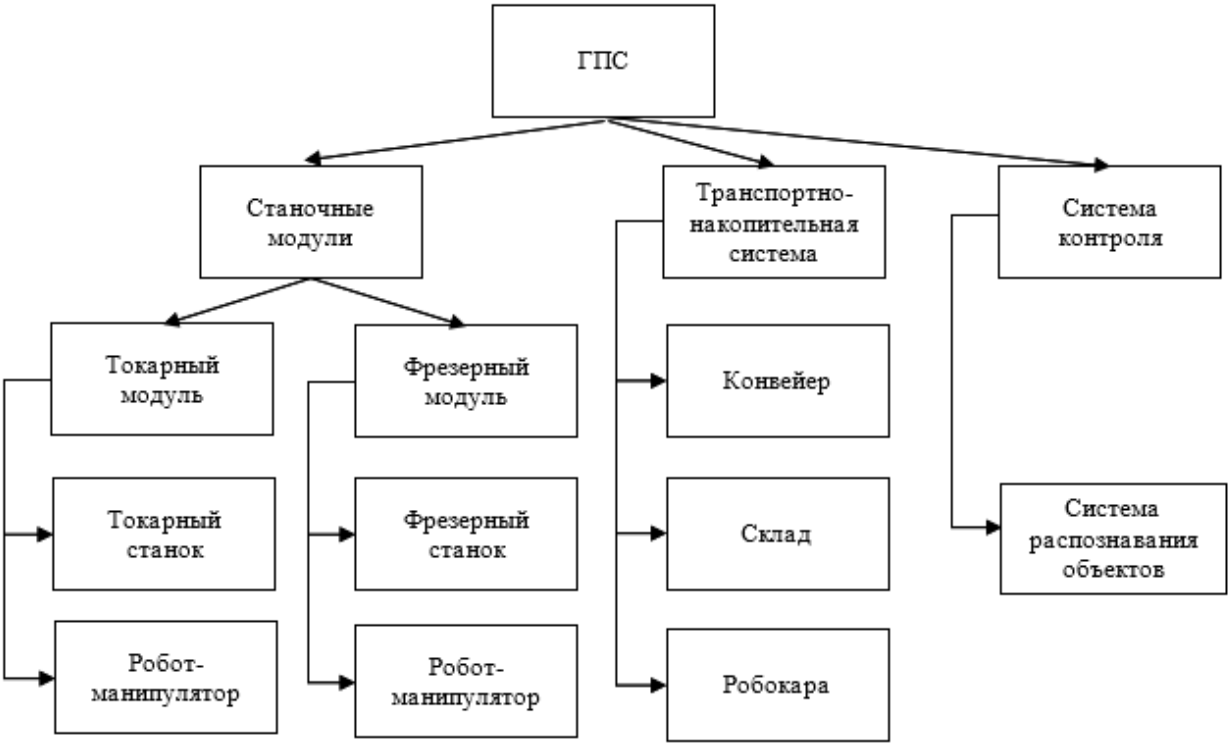


Рисунок 1 – Структурная схема ГПС Denford

Схема расположения модулей ГПС представлена на рисунке 2.

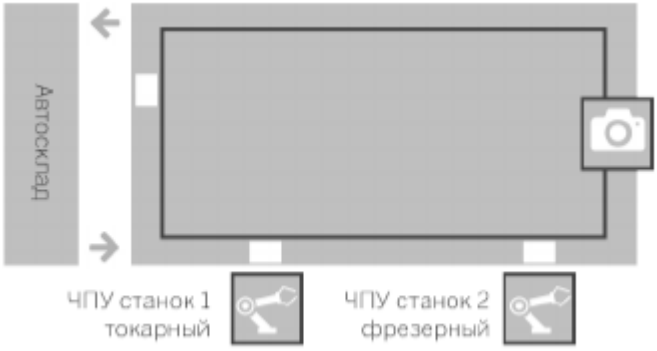


Рисунок 2 – Схема расположения модулей ГПС Denford

В ГПС Denford паллеты движутся против часовой стрелки. В местах, где находятся токарный, фрезерный станки с ЧПУ, система технического зрения Denford и склад, установлены специальные пневматические замки, которые останавливают паллеты с заготовкой, передвигающиеся по ленте конвейера. Каждый замок представляет собой стопор со встроенным ультразвуковым датчиком, срабатывающим на направляющую часть паллеты.

1.2 Анализ принципов работы существующей системы

Система технического зрения состоит из:

- Железного корпуса черного цвета с подставкой для камеры;
- Вмонтированного в корпус экрана, который при подключении к сети загорается белым цветом (рисунок 4) [3];
- Промышленной камера (“Monochrome industrial camera DMK 21AF04”);
- 2 выключателей и адаптера питания. Первый находится на корпусе системы технического зрения, второй – под корпусом рядом с адаптером питания (рисунок 3) [3];
- Компьютера с установленным ПО «Denford Vision System».

Комбинация черного корпуса с белым экраном и монохроматической камерой обеспечивает получение чёткого фото контура объекта для его дальнейшего распознавания.

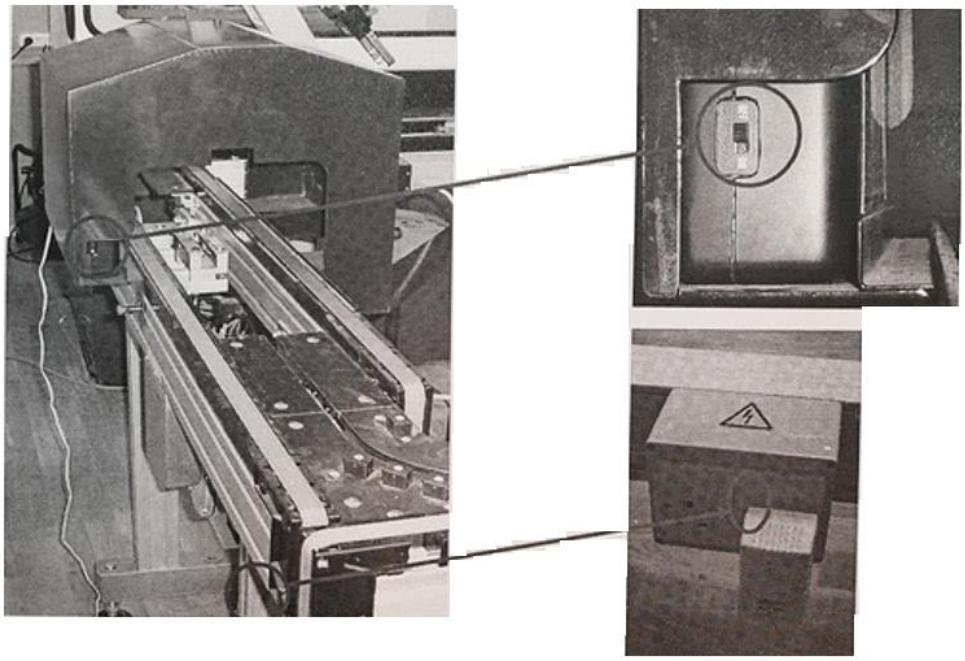


Рисунок 3 – 2 выключателя для включения электропитания

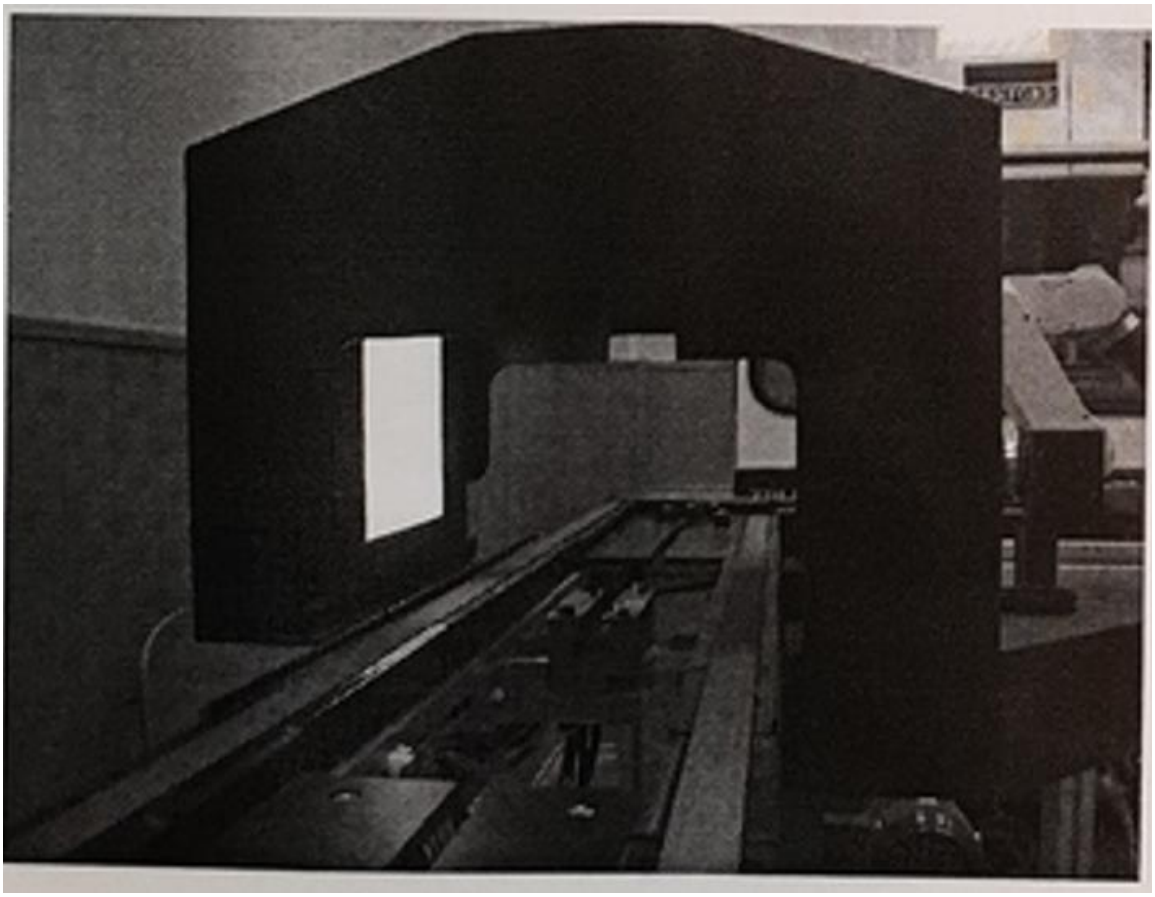


Рисунок 4 – Состояние включенного экрана

В систему технического зрения Denford («Denford Vision System») входят 2 подсистемы (рисунок 5) [3]:

- Система обнаружения объекта «Launch User Interface»;
- Система распознавания объекта «Launch Development Interface».

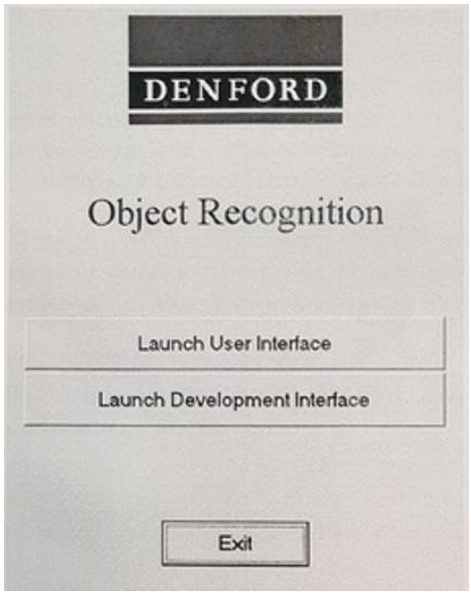


Рисунок 5 – Интерфейс «Denford Vision System»

Принцип работы системы технического зрения:

- Паллета с заготовкой перемещается по ленте и останавливается у системы обнаружения с помощью пневматического замка;
- Оператор включает программу «Launch Development Interface»;
- Оператор устанавливает разрешение фото объекта;
- Оператор производит фотофиксацию (минимум 6 раз с разных углов обзора);
- Фотографию присваивается класс;
- Оператор обводит зону, в которой находится интересующий нас объект;
- Программа тренирует этот класс объекта с помощью этой фотографии;
- Нужно повторить пункты 5-7 со всеми фотографиями;
- Сохранить натренированный класс;
- Оператор включает программу «Launch User Interface»;
- Оператор загружает натренированный класс в программу;
- Оператор производит фотофиксацию объекта, который нужно распознать;
- Система распознает объект.

1.3 Недостатки существующей системы

В исследуемой системе управление техническим зрением осуществляется оператором с помощью программного обеспечения производителя, установленного на сервере устройства (в роли которого выступает ПК с операционной системой Windows XP). Были выявлены следующие недостатки:

- Необходимость оператора для обнаружения объекта;
- Необходимость оператора для идентификации объекта;
- Необходимость остановки паллеты с заготовкой для обнаружения и идентификации объекта;

- Хранение фотографии на ПК с ограниченными возможностями;
- Закрытый исходный код, не позволяющий модернизировать систему;
- Отсутствие подключение к облачной платформе;
- Отсутствует возможность масштабируемости системы (внедрение нового модуля в состав ГПС влечет за собой полное изменение существующей архитектуры системы).

Исходя из этих недостатков можно сформулировать требования к решению:

- Система должна поддерживать автономный режим работы;
- Система должная иметь открытый исходный код, который позволит пользователям внести изменения в зависимости от их поставленной задачи;
- Система должна уметь взаимодействовать с облачной платформой для хранения результатов (в нашем случае с Dropbox);
- Система поддерживает облачное управление техническим зрением;
- Обеспечение низкой стоимости системы, т.к. система учебная.

Таким образом, общая задача курсового проекта – автоматизировать процесс обнаружения объекта.

1.4 Одноплатный компьютер

Одноплатный компьютер – это полноценный компьютер, построенный на одной печатной плате, с несколькими микропроцессорами, памятью, системой ввода/вывода и другими функциями, необходимыми для функционального компьютера. Для системы технического зрения целесообразно использовать именно одноплатные компьютеры в виду низкой стоимости, энергозатраты одноплатных компьютеров значительно меньше. В таблице 1 представлены характеристики имеющихся на рынке одноплатных компьютеров.

Таблица 1 – Характеристики одноплатных компьютеров

Наименование	Цена ($)	Процессор	ОЗУ	LAN	Wireless
B6Duino Zero	39	1 x86 @ 300МГц	128МБ	Да	Нет
A20-OlinuXimo-Lime2	50	1x A7 @ 1ГГц	1Гб	Да	Нет
A20-OlinuXino-Micro	61	1x A7 @ 1ГГц	1Гб	Да	Нет
A33-OlinuXino	40	1x A7 @ 1.2ГГц	1Гб	Нет	Нет
Arduino Industrial 101	39	1x MIPS @ 400МГц	64Мб	Да	WiFi
Arduino Titan	96	1x MIPS @ 560МГц	64Мб	Да	WiFi, BT
Arduino Yun / Yun PoE	68	1x MIPS @ 400МГц	64Мб	Да	WiFi
Да Banana Pi BPI-M2	Да 40	1x A7 @ 1ГГц	1Гб	Да	WiFi
Banana Pi BPI-M2 Ultra	40	4x A7 @ 1ГГц	2Гб	Да	WiFi, BT
Banana Pi BPI-M3	74	8x A7 @ 2ГГц	2Гб	Да	WiFi, BT
Banana Pi BPI-M64	74	4x A53 @ 1.2ГГц	2Гб	Да	WiFi, BT
Banana Pro	48	2x A7 @ 1ГГц	1Гб	Да	WiFi, BT
BeagleBone Black	48	1x A8 @ 1ГГц	512Мб	Да	Нет
BeagleBone Black Wireless	69	1x A8 @ 1ГГц	512Мб	Нет	WiFi, BT
BeagleBone Blue	80	1x A8 @ 1ГГц	512Мб	Нет	WiFi, BT
BeagleBone Green	39	1x A8 @ 1ГГц	512Мб	Да	Нет

Продолжение таблицы 1

BeagleBone Green Wireless	50	1x A8 @ 1ГГц	512Мб	Да	WiFi, BT
Bubblegum-96	89	4x A53 @ 1.8ГГц	2Гб	Нет	WiFi, BT
ChipPro Dev Kit	49	1x A8 @ 1ГГц	256Мб	Нет	WiFi, BT
CloudBit	60	1x ARM9 @ 454МГц	64Мб	Нет	WiFi
Creator CI40	85	2x MIPS @ 550МГц	256Мб	Да	WiFi, BT
CubieAIO-A20	122	2x A7 @ 1ГГц	1Гб	Да	WiFi, BT
Cubieboard4	135	4x A15 @ 2ГГц + 4x A7 @ 1.3ГГц	2Гб	Да	WiFi, BT
Cubieboard5	108	8x A7 @ 2ГГц	2Гб	Да	WiFi, BT
DE0-Nano-SoC Dev Kit	99	2x A9 @ 952МГц + FPGA	1Гб	Да	Нет
DPT-Board	56	1x MIPS @ 400МГц	64Мб	Да	WiFi
DragonBoard 410c	75	4x A53 @ 1.2ГГц	1Гб	Нет	WiFi, BT
Edison Kit for Arduino	110	2x x86 @ 500МГц + Quark	1Гб	Нет	WiFi, BT
Firefly-RK3288	129	4x A17 @ 1.8ГГц	2Гб	Да	WiFi, BT
Firefly-RK3288 Reload	159	4x A17 @ 1.8ГГц	2Гб	Да	WiFi, BT
Firefly-RK3399	179	2x A72 @ 2ГГц, 4x A53 @ 1.4ГГц	2Гб	Да	WiFi, BT
Galileo Gen 2	62	1x x86 @ 400МГц	256Мб	Да	Нет
Gizmo 2	165	2x x86 @ 1ГГц	1Гб	Да	Нет

Продолжение таблицы 1

HiKey	75	8x A53 @ 1.2ГГц	1Гб	Нет	WiFi, BT
HummingBoard-Base	74	1x A9 @ 1.2ГГц	512Мб	Да	Нет
HummingBoard-Edge	99	1x A9 @ 1.2ГГц	512Мб	Да	Нет
HummingBoard-Gate	84	1x A9 @ 1.2ГГц	512Мб	Да	Нет
HummingBoard-Pro	84	1x A9 @ 1.2ГГц	512Мб	Да	Нет
Inforce 6410Plus	143	4x A15 @ 1.7ГГц	2Гб	Да	WiFi, BT
Khadas Vim	50	4x A53 @ 2.0ГГц	2Гб	Да	WiFi, BT
LeMaker Guitar	59	4x A9 @ 1.6ГГц	1Гб	Да	WiFi, BT
Linkit Smart 7688	13	1x MIPS @ 580МГц	128Мб	Нет	WiFi
LinkSprite Acadia V3	119	4x A9 @ 1.2ГГц	1Гб	Да	Нет
LinkSprite Arches	95	4x A15 @ 2ГГц + 4x A7 @ 1.3ГГц	2ГБ	Да	WiFi, BT
MarsBoard RK3066 Pro	140	4x A9 @ 1.6ГГц	1Гб	Да	Нет
MediaTek X20 Dev Board	199	2x A72 @ 2.5ГГц + 4x A53 @ 1.9ГГц + 4x A53 @ 1.4ГГц	2Гб	Нет	WiFi, BT
MinnowBoard Turbot	140	2x x86 @ 1.46ГГц	2Гб	Да	Нет
MinnowBoard Turbo Quad	190	2x x86 @ 1.46ГГц	2Гб	Да	Нет
MYS-6ULX SBC	25	1x Cortex-A7 @ 528MGz	256Мб	Да	WiFi

Продолжение таблицы 1

NanoPC-T3	60	8x A53 @ 1.4ГГц	1Гб	Да	WiFi, BT
NanoPi A64	25	8x A53 @ 1.2ГГц	1Гб	Да	Нет
NanoPi K2	40	8x A53 @ 1.5ГГц	2Гб	Да	WiFi, BT
NanoPi M1	15	4x A7 @ 1.2ГГц	512Мб	Да	Нет
NanoPi M1 Plus	35	4x A7 @ 1.2ГГц	1Гб	Да	WiFi, BT
NanoPi M2A / M3	30	4x A53 @ 1.4ГГц	1Гб	Да	WiFi, BT
NanoPi2 Fire	23	4x A9 @ 1.4ГГц	1Гб	Да	Нет
NanoPi Neo / Neo2	15	4x A7 @ 1.2ГГц	256Мб	Да	Нет
NanoPi Neo Air	20	4x A7 @ 1.2ГГц	512Мб	Нет	WiFi, BT
NanoPi S2	45	4x A9 @ 1.4ГГц	1Гб	Нет	WiFi, BT
Odroid-C0	31	4x A5 @ 1.5ГГц	1Гб	Нет	Нет
Odroid-C1+	35	4x A5 @ 1.5ГГц	1Гб	Да	Нет
Odroid-C2	46	4x A53 @ 1.5ГГц	2Гб	Да	Нет
Odroid-XU4	59	4x A15 @ 2ГГц + 4x A7 @ 1.4ГГц	2Гб	Да	Нет
Orange Pi 2G-IoT	10	1x Cortex-A5	256Мб	Нет	WiFi, BT
Orange Pi Lite	12	4x A7 @ 1.2ГГц	512Мб	Нет	WiFi
Orange Pi One	10	4x A7 @ 1.2ГГц	512Мб	Да	Нет

Продолжение таблицы 1

Orange Pi PC / PC Plus	15	4x A7 @ 1.6ГГц	1Гб	Да	Нет
Orange Pi PC 2	20	4x A53 @ 1.4ГГц	1Гб	Да	Нет
Orange Pi Plus2 / Plus2E	49	4x A7 @ 1.6ГГц	2Гб	Да	WiFi
Orange Pi Prime	30	4x A53 @ 1.4ГГц	2Гб	Да	WiFi, BT
Orange Pi Win Plus / Win	30	4x A53 @ 1.4ГГц	2Гб	Да	WiFi, BT
Orange Pi Zero	9	4x A7 @ 1.2ГГц	256Мб	Да	WiFi
Orange Pi Zero Plus 2 H3 / H5	19	4x A7 @ 1.6ГГц	512Мб	Нет	WiFi, BT
Parallella	99	2x A7 @ 667МГц + FGPA	1Гб	Да	Нет
pcDuino Lite WiFi	29	1x A8 @ 1ГГц	512Мб	Да	Нет
pcDuino8 UHет	49	8x A7 @ 2ГГц	1Гб	Да	Нет
Pepper 43R / 43C	169	1x A8 @ 800МГц	512Мб	Да	WiFi, BT
Pepper DVI-D	119	1x A8 @ 800МГц	512Мб	Да	Нет
PICO-IMX6UL-KIT	65	1x A7 @ 528МГц	512Мб	Да	WiFi, BT
Pine A64	15	4x A53 @ 1.2ГГц	512Мб	Да	Нет
Raspberry Pi Zero	14	1x ARM11 @ 1ГГц	512Мб	Нет	Нет
Raspberry Pi Zero W	19	1x ARM11 @ 1ГГц	512Мб	Нет	WiFi, BT
Raspberry Pi 2 Model B	35	4x A53 @ 900МГц	1Гб	Да	Нет

Продолжение таблицы 1

Raspberry Pi 3 Model B	35	4x A53 @ 1.2ГГц	1Гб	Да	WiFi, BT
Rico Board	99	1x A9 @ 1ГГц	512Мб	Да	Нет
RioTboard	79	1x A9 @ 1ГГц	1Гб	Да	Нет
SAMA5D4 Xplained	94	1x A5 @ 528МГц	512Мб	Да	Нет
SavageBoard	59	1x A9 @ 1ГГц	512Мб	Да	BT
Seeeduino Cloud	70	1x MIPS @ 400МГц	64Мб	Да	WiFi
Tinker Board	60	4x Cortex-A17 @ 1.8ГГц	2Гб	Да	WiFi, BT
Udoo Neo	50	1x A9 @ 1ГГц	512Мб	Да	Нет
Udoo Quad / Dual / Dual Basic	99	2x A9 @ 1ГГц	1Гб	Нет	Нет
Udoo X86	125	4x Brawswell @ 2.56ГГц	2Гб	Да	WiFi, BT
UP board	89	4x Cherry Trail @ 1.1ГГц	1Гб	Да	Нет
UP Squared	145	2x Apollo Lake @ 1.1ГГц	2Гб	Да	Нет
USB Armory	123	1x A8 @ 800МГц	512Мб	Нет	Нет
Wandboard Reload	79	1x A9 @ 1ГГц	512Мб	Да	Нет
Z-turb Board	99	2x ARM @ 667МГц + FGPA	1Гб	Да	Нет

2. Концептуальное проектирование системы

2.1 Постановка задачи

Недостатки, выделенные в системе, решаются путем налаживания управления производством из облачных сервисов по протоколу интернета вещей. Таким образом, возможно не просто исключить человека из производственного процесса, повышая безопасность, так как человеку не придется находится в рабочих зонах различных роботов, станков манипуляторов, но и дать возможность управления ГПС из любой точки мира, где есть выход в сеть. Управляющие программы будут разрабатываться на современных языках, что позволит актуализировать ПО.

В настоящее время существует множество мощных методов для решения задачи обнаружения идентификации объекта, но не все подходят для поставленной задачи из-за того, что они являются достаточно дорогостоящими в плане вычисления. Нужно также учитывать такие параметры, как расстояние от камеры до объекта, условие освещения, отражение, возможные изменения в окружающей среде, влияющий на фон изображения, устройство, на котором написана программа и другие, потому что они могут сбить с толку алгоритмы. В связи с этим нужно осторожно подойти к вопросу выбора решения задачи обнаружения объекта.

На данный момент на рынке существует множество облачных сервисов, которые позволяют системе распознавания объекта управлять, хранить, передавать и делиться с другими пользователями фото, полученное в результате работы программы, поэтому появляется необходимость выбрать сервис, который позволяет системе быстро и удобно взаимодействовать с облаком.

На основании вышесказанного была сформулирована цель выпускной квалификационной работы.

Цель курсового проекта – разработать систему распознавания движущихся производственных объектов в облачной системе управления.

В ходе проекта были сформулированы следующие задачи:

- проанализировать существующую систему распознавания и выявить ее недостатки;
- на основе анализа полученных данных сформулировать требования, предъявляемые к разрабатываемой системе управления;
- разработать функциональную модель распознавания движущихся объектов;
- разработать алгоритмы основных функций программ;
- разработать программу для взаимодействия Raspberry Pi и веб-приложения по сети Интернет через протокол MQTT от облачной платформы Yandex.Cloud.

2.2 Разработка функциональной диаграммы системы распознавания

В ходе работы над проектом по созданию системы распознавания движущихся объектов в облачной системе управления была разработана функциональная диаграмма распознавания движущихся объектов в автоматическом режиме с использованием нотации IDEF0, которая показана на рисунке 6.

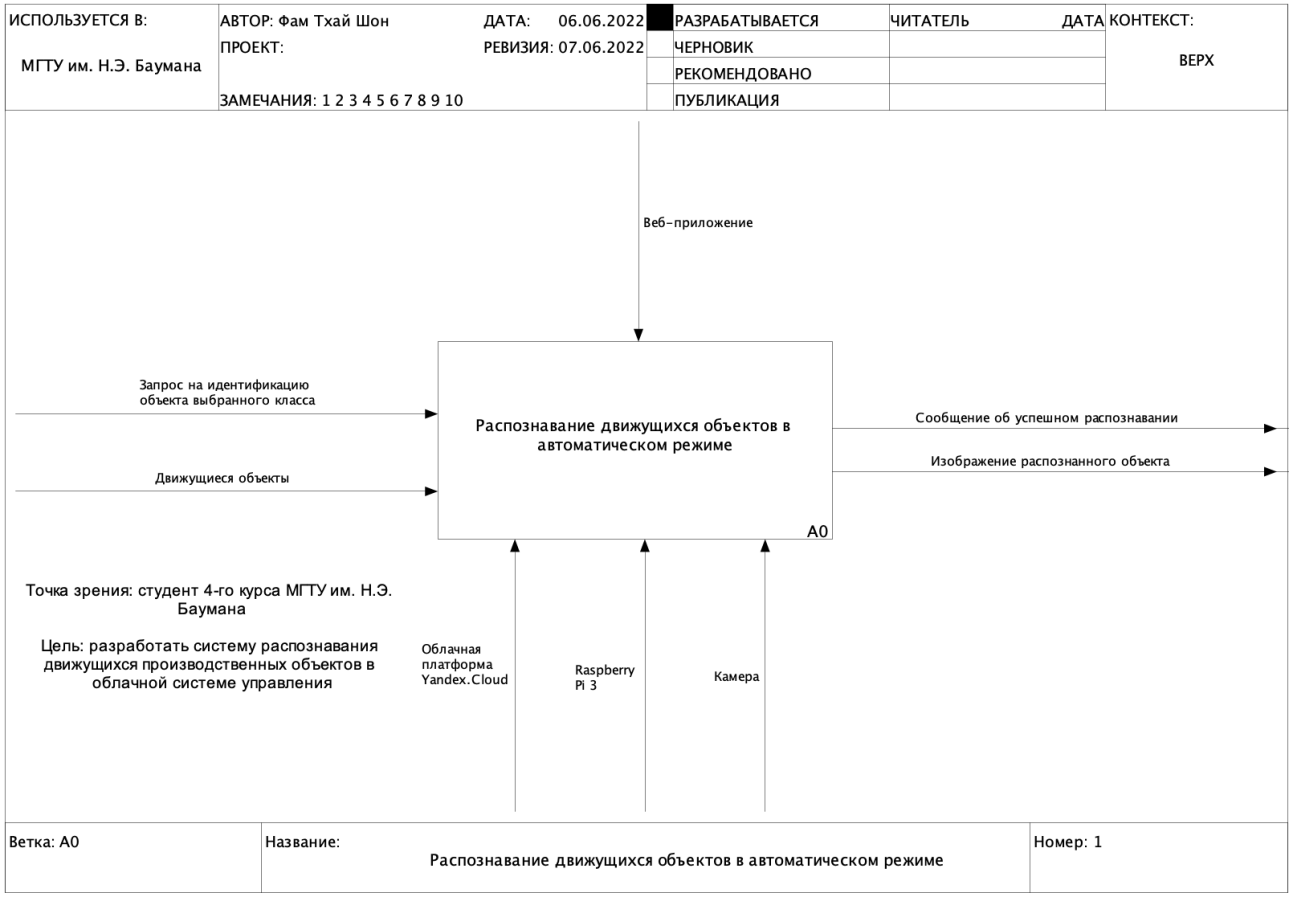


Рисунок 6 – Функциональная модель распознавания движущихся объектов в автоматическом режиме (блок A0)

Функциональная модель разрабатываемой системы отражает этапы распознавания движущихся производственных объектов в автоматическом режиме, начиная с получения управляющей команды сервером устройства от облачной платформы и заканчивая получением результата на сервере устройств (рисунок 7).

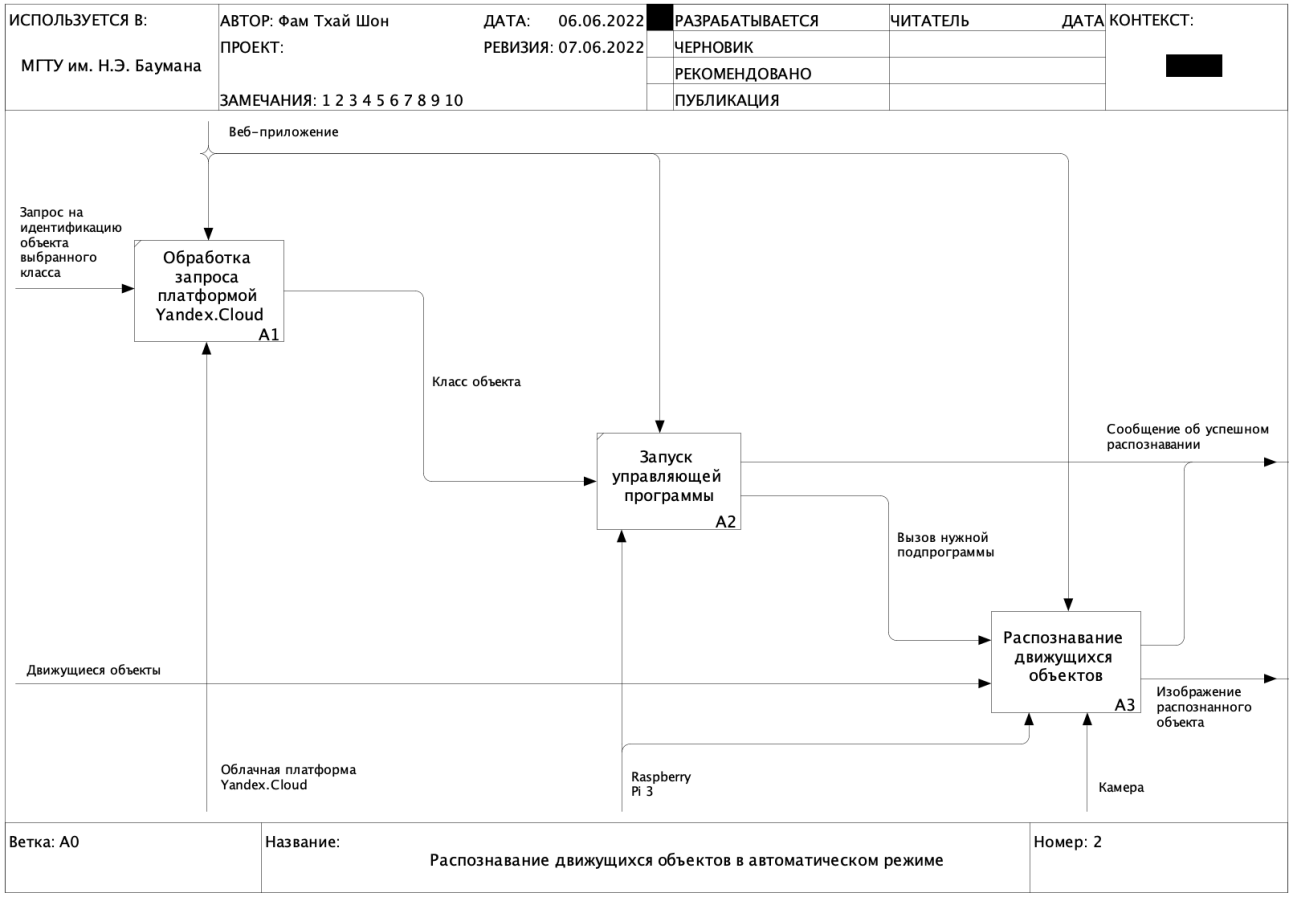


Рисунок 7 – Функциональная модель распознавания движущихся объектов в автоматическом режиме (блок A0)

Процесс распознавания движущихся производственных объектов начинается с обработки облачной платформой запроса на поиск объекта определенного класса, который пользователь выбрал на веб-приложении, далее платформа Yandex.Cloud отправляет команду на поиск объекта Raspberry Pi. Сервер устройства с помощью управляющей программы обрабатывает полученную команду и запускает необходимую подпрограмму. Запущенная подпрограмма вызывает алгоритм распознавания движущихся производственных объектов того класса, который был выбран пользователем. Используя библиотеку Picamera и модуль камеры IMX219 8MP Arducam, подключенный к Raspberry Pi с помощью шлейфа, программа получает доступ к видеопотоку, происходит фотофиксация, обнаружение и идентификация объекта. При успешном распознавании объекта выбранного класса Raspberry Pi отправляет результат программы (фото с распознанным объектом и сообщение следующего формата:” Название класса объекта” was found) на веб-приложение,

а также отправляет фото с результатом на Dropbox для хранения. Если проезжает объект, но не того класса, который был выбран пользователем, то программа работает дальше.

Этап распознавания объекта был разобран более подробно. Функциональная диаграмма представлена на рисунке 8.

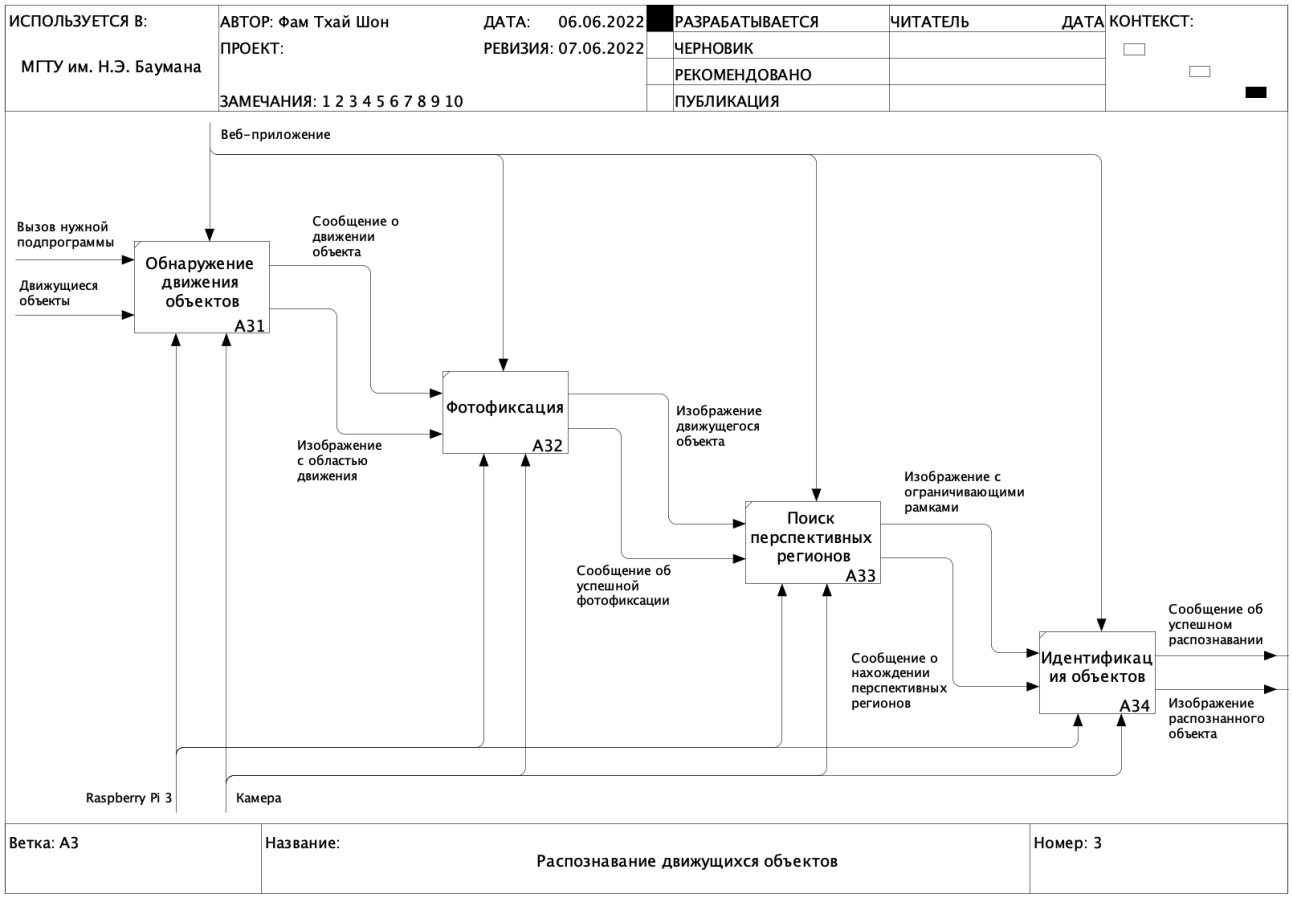


Рисунок 8 – Функциональная модель распознавания движущихся объектов в автоматическом режиме (блок A3)

2.3 Разработка архитектуры системы распознавания объектов в облачной системе управления

Для наглядного представления архитектуры системы распознавания движущихся производственных объектов в облачной системе управления используется диаграмма, представленная на рисунке 9.

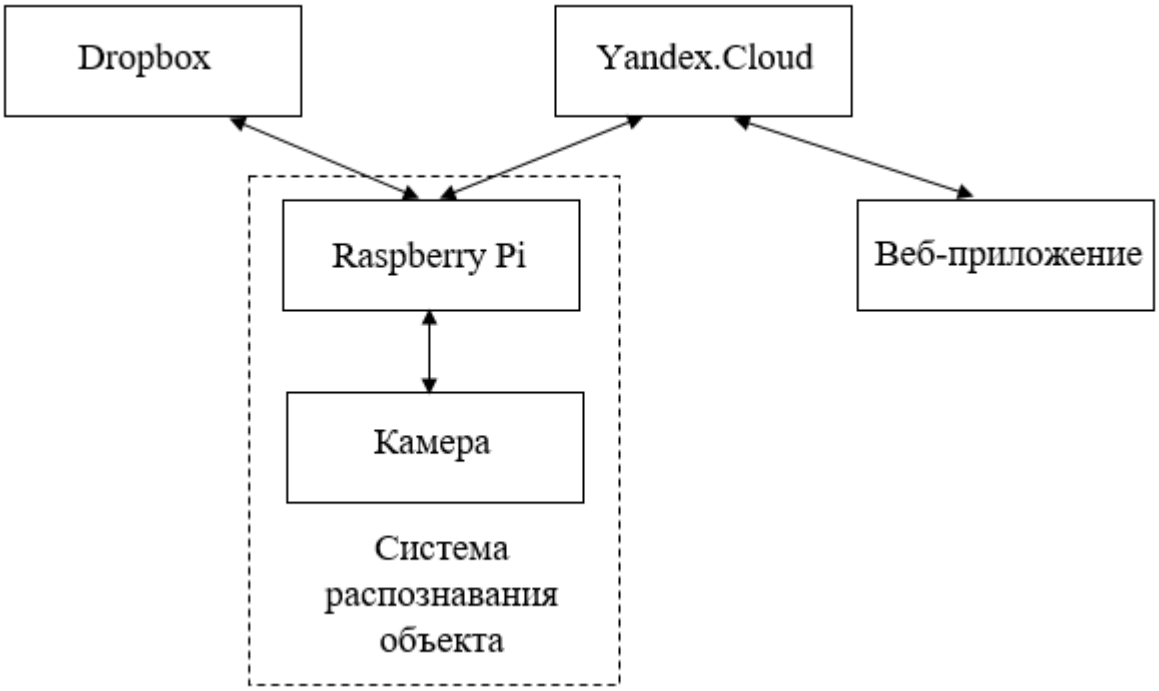


Рисунок 9 – Диаграмма архитектуры системы распознавания в облачной системе управления

В предлагаемом решении проектируемая система состоит из:

- веб-приложения;
- облачной платформы Yandex.Cloud;
- файлового хостинга Dropbox;
- одноплатного компьютера Raspberry Pi model B;
- модуля камеры IMX219 8MP Arducam для Raspberry Pi.

Веб-приложение уже был развернут другим студентом для управления различных модулей ГПС по сети Интернет через протокол MQTT от облачной платформы Yandex.Cloud. Одноплатный компьютер Raspberry Pi 3 model B, способный свободно выходить в сеть Интернет, что позволяет взаимодействовать с веб-приложением, и в то же время с модулем камеры IMX219 8MP Arducam с помощью шлейфа и библиотеки OpenCV и Picamera. В свою очередь модуль камеры IMX219 8MP Arducam предоставляет доступ к видеопотоку одноплатному компьютеру Raspberry Pi 3. Файловый хостинг Dropbox выступает в качестве хранилища результатов распознавания объектов и

взаимодействует с Raspberry Pi с помощью библиотеки dropbox. Облачная платформа Yandex.Cloud необходима как «канал взаимодействия» между объектами гибкой производственной системы и пользователем. Она позволяет пользователю управлять объектами производственной системы в любом месте, где доступен Интернет.

Но не стоит списывать старую систему технического зрения, потому что была создана хорошая аппаратная часть в лице черного корпуса с подставкой для камеры, вмонтированного в корпус экрана с белой подцветкой. Это хорошее сочетание, которое помогает камеры четко увидеть предмет, и в то же время способствует программе распознавания работать быстрее за счет черно-белого фона. Ниже представлена более подробная архитектура аппаратной части (рисунок 10).

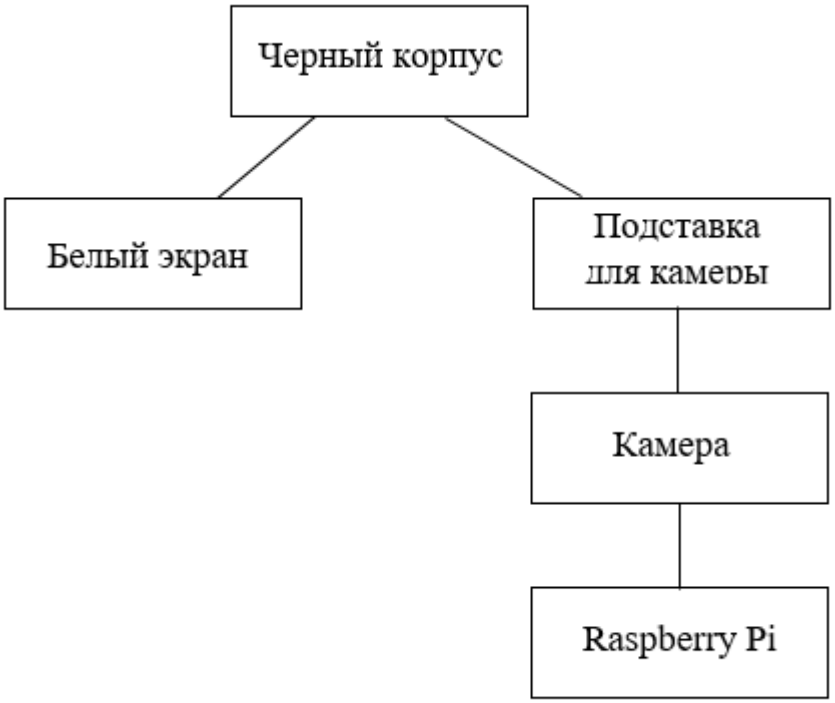


Рисунок 10 – Диаграмма архитектуры аппаратной части системы распознавания объектов

Аппаратная часть состоит из:

- железного черного корпуса;

- подставки для камеры;
- белого экрана, вмонтированного в корпус;
- модуля камеры IMX219 8MP Arducam для Raspberry Pi;
- Raspberry Pi 3 model B.

Сочетание черного корпуса с белым экраном представляет собой хорошей комбинацией, позволяющая камере четко увидеть предмет и программе распознавания работать быстрее за счет черно-белого фона. Подставка для камеры является частью железного корпуса и позволяет фиксировать камеру на одном месте, несмотря на вибрацию от работы конвейера.

3. Техническое задание

3.1 Общие сведения

Полное наименование системы и ее условное обозначение

Система распознавания движущихся производственных объектов в облачной системе управления.

Наименование предприятий (объединений) разработчика и заказчика (пользователя) системы и их реквизиты

Разработчик:

Фам Тхай Шон

Предприятие:

Группа РК9и-85Б

Заказчик: МГТУ им. Н.Э. Баумана

Юридический адрес: г. Москва, 2-я Бауманская улица, 5 стр.1

Плановые сроки работы по созданию системы

Дата начала работы – 01.09.2021 г.

Дата окончания работы – 30.05.2022 г.

3.2 Назначение и цели создания системы

Назначение системы

Разрабатываемая система предназначена для распознавания движущихся производственных объектов и ее контроля на облачной платформе.

Цели создания системы

Система распознавания движущихся производственных объектов в облачной системе управления позволяет решать следующие задачи:

- получение возможности удалённого управления и контроля за процессом распознавания движущихся объектов удаленно;

- интеграция модуль контроля в состав гибкой производственной системы;
- обмен данными между облачной платформой, камерой и одноплатным компьютером Raspberry Pi 3;
- идентификация объектов во время движения;
- увеличение производительности за счет распознавания движущихся объектов в реальном времени;
- повышение удобства эксплуатации системы.

3.3 Требования к системе в целом

Требования к функционированию системы:

- система распознавания движущихся объектов должна поддерживать как автономный режим работы, так и режим управления оператором ГПС;
- система должна однозначно определять движущийся производственный объект в реальном времени;
- система управления должна взаимодействовать с облачной платформой Yandex.Cloud и файловым хостингом Dropbox;
- программная часть системы распознавания движущихся объектов должна иметь открытый исходный код, который позволит пользователям внести изменения в зависимости от поставленных целей;
- исходный код программной части системы управления, отвечающий за взаимодействие с облачной платформой, должен быть инкапсулирован в отдельный файл программы.

Требования к структуре системы:

Структура системы распознавания движущихся производственных объектов в облачной системе управления состоит из трех уровней:

- нижний уровень состоит из исполнительного устройства (модуля камеры IMX219 8MP Arducam), железного корпуса черного цвета с подставкой для камеры, экрана, который был вмонтирован в корпус и загорается белым цветом при подключении к сети;
- средний уровень состоит из одноплатного компьютера Raspberry Pi 3; сервер устройства системы распознавания обеспечивает взаимодействие между модулем камеры и облачной платформой (верхний уровень);
- верхний уровень состоит из Web-сервера, развернутого на облачной платформе Yandex.Cloud и обеспечивающего контроль процессом распознавания движущихся производственных объектов, файлового хостинга Dropbox для хранения результатов работы программы.

Техническое задание разработано в соответствии с ГОСТ 34.602-89 [4].

4. Структурное проектирование

На этом этапе проводятся сравнительные анализы существующих технических решений и выборы наиболее подходящего из них, удовлетворяющего требованиям, предъявляемым к разрабатываемой системе. Также нужно сделать выбор протоколов и интерфейсов для взаимодействия элементы системы распознавания движущихся производственных объектов в системе управления с файловым хостингом Dropbox и облачной платформой Yandex.Cloud, разработать основные алгоритмы распознавания объектов и набор команд, позволяющий веб-приложению и одноплатному компьютеру обмениваться информацией с платформой Yandex.Cloud.

4.1 Сравнительный анализ и выбор одноплатного компьютера

По требованию обеспечения низкой стоимости системы в целом, так и её компонентов, исключаем персональные компьютеры из вариантов реализации аппаратной части. В таблице 1 настоящей работы представлены характеристики имеющихся на рынке одноплатных компьютеров. Исходя требования минимизации стоимости, были выбраны все одноплатные компьютеры стоимостью ниже 40$, но все же для хорошей работы системы необходимо, чтобы объем ОЗУ был более 512 Мб и мог подключаться к интернету для работы с облачными платформами. Исходя из этого, из одноплатных компьютеров, представленных в таблице 2 и удовлетворяющих изложенным требованиям, можно выделить:

- NanoPi M2A / M3 стоимостью 30$;
- Orange Pi Prime стоимостью 30$;
- Orange Pi Win Plus / Win стоимостью 30$;
- NanoPi M1 Plus стоимостью 35$;

- Raspberry Pi 3 Model B стоимостью 35$.

Можно было бы использовать любой одноплатный компьютер из представленных стоимостью 30$. Но мы остановим свой выбор на Raspberry Pi 3 Model B. Для работы с ним и изучения была использована его документация [5]. Выбор основывается на том, что эти компьютеры уже имелись в имуществе нашей кафедры. Так же Raspberry Pi успешно применяется в автоматизированном производстве для решения разных задач.

Таблица 2 – Одноплатные компьютеры стоимостью менее 40$

Наименование	Цена ($)	Процессор	ОЗУ	LAN	Wireless
Orange Pi Zero	9	4x A7 @ 1.2ГГц	256Мб	Да	WiFi
Orange Pi 2G-IoT	10	1x Cortex-A5	256Мб	Нет	WiFi, BT
Orange Pi One	10	4x A7 @ 1.2ГГц	512Мб	Да	Нет
Orange Pi Lite	12	4x A7 @ 1.2ГГц	512Мб	Нет	WiFi
Linkit Smart 7688	13	1x MIPS @ 580МГц	128Мб	Нет	WiFi
Raspberry Pi Zero	14	1x ARM11 @ 1ГГц	512Мб	Нет	Нет
NanoPi M1	15	4x A7 @ 1.2ГГц	512Мб	Да	Нет
NanoPi Neo / Neo2	15	4x A7 @ 1.2ГГц	256Мб	Да	Нет
Orange Pi PC / PC Plus	15	4x A7 @ 1.6ГГц	1Гб	Да	Нет
Pine A64	15	4x A53 @ 1.2ГГц	512Мб	Да	Нет
Orange Pi Zero Plus 2 H3 / H5	19	4x A7 @ 1.6ГГц	512Мб	Нет	WiFi, BT
Raspberry Pi Zero W	19	1x ARM11 @ 1ГГц	512Мб	Нет	WiFi, BT
NanoPi Neo Air	20	4x A7 @ 1.2ГГц	512Мб	Нет	WiFi, BT
Orange Pi PC 2	20	4x A53 @ 1.4ГГц	1Гб	Да	Нет
NanoPi2 Fire	23	4x A9 @ 1.4ГГц	1Гб	Да	Нет

Продолжение таблицы 2

pcDuino Lite WiFi	29	1x A8 @ 1ГГц	512Мб	Да	Нет
NanoPi M2A / M3	30	4x A53 @ 1.4ГГц	1Гб	Да	WiFi, BT
Orange Pi Prime	30	4x A53 @ 1.4ГГц	2Гб	Да	WiFi, BT
Orange Pi Win Plus / Win	30	4x A53 @ 1.4ГГц	2Гб	Да	WiFi, BT
Odroid-C0	31	4x A5 @ 1.5ГГц	1Гб	Нет	Нет
NanoPi M1 Plus	35	4x A7 @ 1.2ГГц	1Гб	Да	WiFi, BT
Odroid-C1+	35	4x A5 @ 1.5ГГц	1Гб	Да	Нет
Raspberry Pi 2 Model B	35	4x A53 @ 900МГц	1Гб	Да	Нет
Raspberry Pi 3 Model B	35	4x A53 @ 1.2ГГц	1Гб	Да	WiFi, BT
B6Duino Zero	39	1 x86 @ 300МГц	128МБ	Да	Нет
Arduino Industrial 101	39	1x MIPS @ 400МГц	64Мб	Да	WiFi
BeagleBone Green	39	1x A8 @ 1ГГц	512Мб	Да	Нет

4.2 Взаимодействие Raspberry Pi 3 с веб-приложением через облачную платформу Yandex.Cloud

В данном проекте система распознавания движущихся объектов разрабатывается на основе одноплатного компьютера Raspberry Pi 3. Этот сервер устройства подключается к сети Интернет, затем через протокол MQTT от Yandex.Cloud передает и получает сообщения от веб-приложения (рисунок 11).

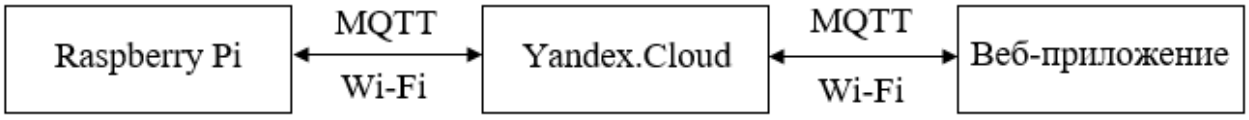


Рисунок 11 – Схема взаимодействия устройств

Протокол MQTT (Message Queue Telemetry Transport) – это легкий, компактный и открытый протокол обмена данными, созданный для передачи информации на удалённых локациях, где объем передаваемых данных невелик и есть ограничения по пропускной способности канала [6]. Основные превосходства протокола MQTT над другими:

- компактные передаваемые сообщения;
- работа при нестабильном соединении;
- легкая интеграция новых устройств;
- поддержка нескольких уровней качества обслуживания.

Протокол MQTT работает на прикладном уровне поверх протокола транспортного уровня TCP и протокола сетевого уровня IP (рисунок 12) [6].

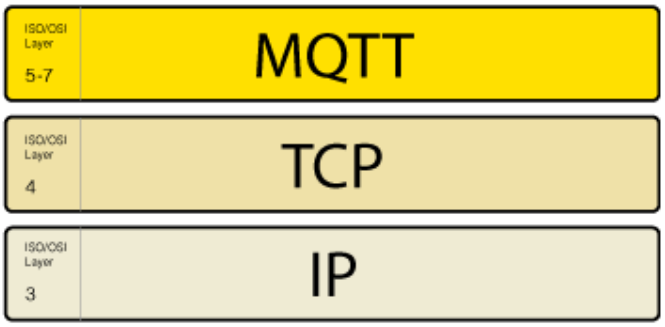


Рисунок 12 – Уровни протоколов передачи данных

Обмен сообщениями в протоколе MQTT осуществляется между клиентом («client»), который может быть издателем («publisher») или подписчиком («subscriber») сообщений, и брокером («broker») сообщений. Платформа «Yandex Cloud» предоставляет доступ к собственному MQTT-брокеру по адресу mqtt.cloud.yandex.net, используя 8883 порт для подключения [7].

Издатель отправляет данные на MQTT-брокер, указывая в сообщении определенный топик. Подписчики данного топика могут получать разные данные от множества издателей, также один издатель может публиковать сообщения в множество топиков.

Устройства MQTT используют определенные типы сообщений для взаимодействия с брокером, ниже представлены основные:

- «connect» – установить соединение с брокером;
- «disconnect» – разорвать соединение с брокером;
- «publish» – опубликовать данные в топик на брокере;
- «subscribe» – подписаться на топик на брокере;
- «unsubscribe» – отписаться от топика.

Взаимодействие между издателем, брокером и клиентом был представлен на UML-диаграмме последовательности (рисунок 13).

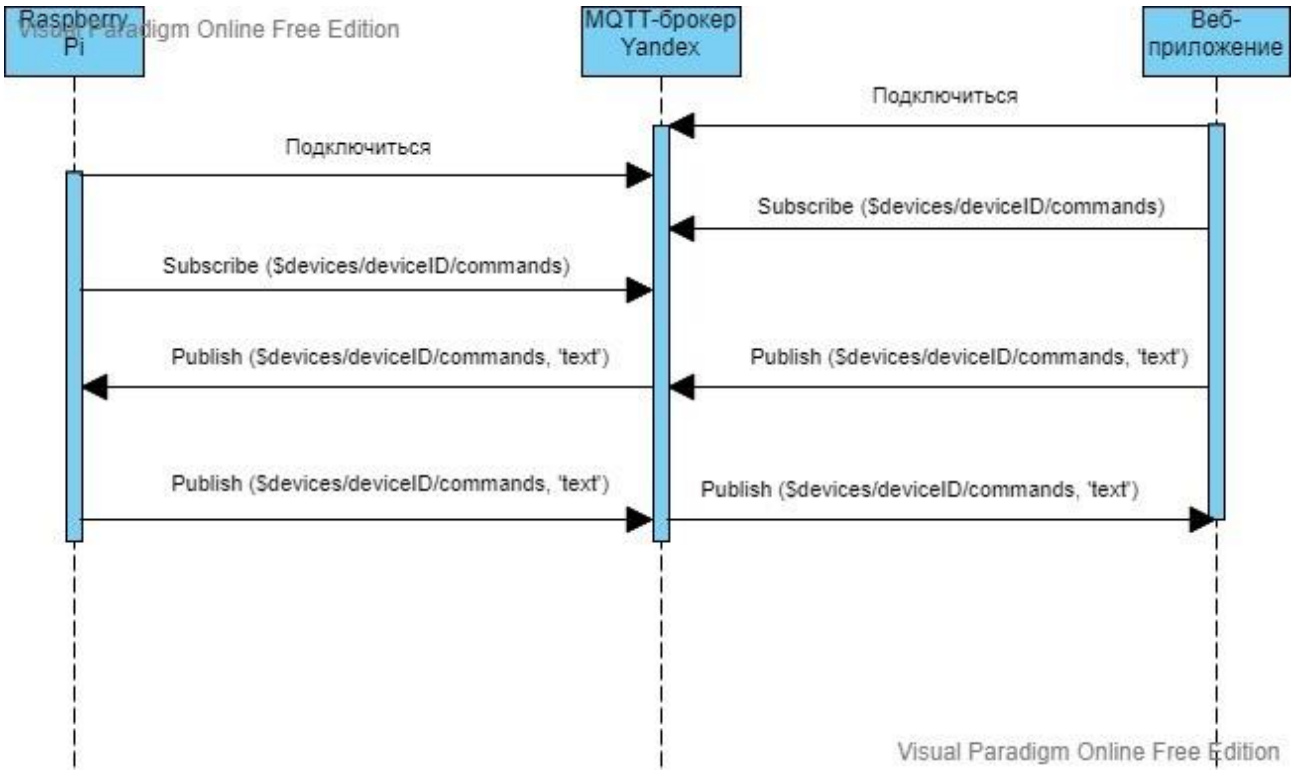


Рисунок 13 – UML-диаграмма последовательности взаимодействия сервера устройства с веб-приложением через облако

В начале создается TCP/IP соединение между сервером устройства и облачной платформой Yandex Cloud. Затем сервер устройства подключается к MQTT-брокеру, получает подтверждение подключения и подписывается на определенный топик брокера Yandex.Cloud.

В данной работе для подписки клиента на определенный топик используется следующий формат сообщения: «$devices/deviceID/commands», где «deviceID» – идентификационный номер устройства, присвоенный ему платформой «Yandex Cloud» при регистрации в облачном сервисе, «commands» – название топика, в который MQTT-брокер публикует команды [7].

В данной системе веб-приложение и одноплатный компьютер Raspberry Pi выступают одновременно в роли издателя и подписчика. Пользователь выбирает класс объекта, который нужно найти и идентифицировать, на сайте. В этом случае в роли издателя выступает веб-приложение, после подключения к MQTT-брокеру, подтверждает подключение, отправляет сообщение, содержащее название класса объекта, который нужно распознать в топик «/commands», на который подписан Raspberry Pi. После публикации сообщения издателем в

определенный топик все подписчики данного топика получают эту информацию. При успешном распознавании объекта в качестве издателя выступает Raspberry Pi, и он отправляет результат работы алгоритма с сообщением, что объект выбранного класса был найден.

4.3 Формат сообщений между облачной платформой Yandex.Cloud и Raspberry Pi 3

Обмен данными между одноплатным компьютером, управляющим камерой, и облачной платформой Yandex.Cloud осуществляется с помощью следующих 2 типов сообщений.

Первый тип сообщений – сообщения, передающие информацию об успешном распознавании объекта:

- «была обнаружена X» – Raspberry Pi передает сообщение об успешном распознавании объекта.

Второй тип сообщений – управляющие команды, которые передает облачная платформа одноплатному компьютеру. Управляющая команда:

- Команда на распознавания объектов определенного класса, который выбирает пользователь на веб-приложении, формат таких сообщений следующий: «Обнаружить X», где X – название класса производственного объекта, создается оператором ГПС при наладке всей системы и используется для однозначной идентификации паллеты при работе склада в автоматическом режиме.

4.4 Взаимодействие Raspberry Pi 3 с модулем камеры

Взаимодействие сервера устройства склада, в роли которого выступает Raspberry Pi 3 с ОС Ubuntu, с модулем камеры осуществляется через шлейф.

На одноплатном компьютере Raspberry Pi 3 и на модуле камеры IMX219 расположены CSI разъёмы, которые предназначены для шлейфа. На рисунке 14 представлен этот разъём.

Рисунок 14 – CSI разъём для модуля камеры на Raspberry Pi 3

Одноплатный компьютер получает доступ к модулю камеры, подсоединив шлейф к этим разъемам. Распиновка разъема камеры и ее схема представлены на рисунке 15 и в таблице 3.

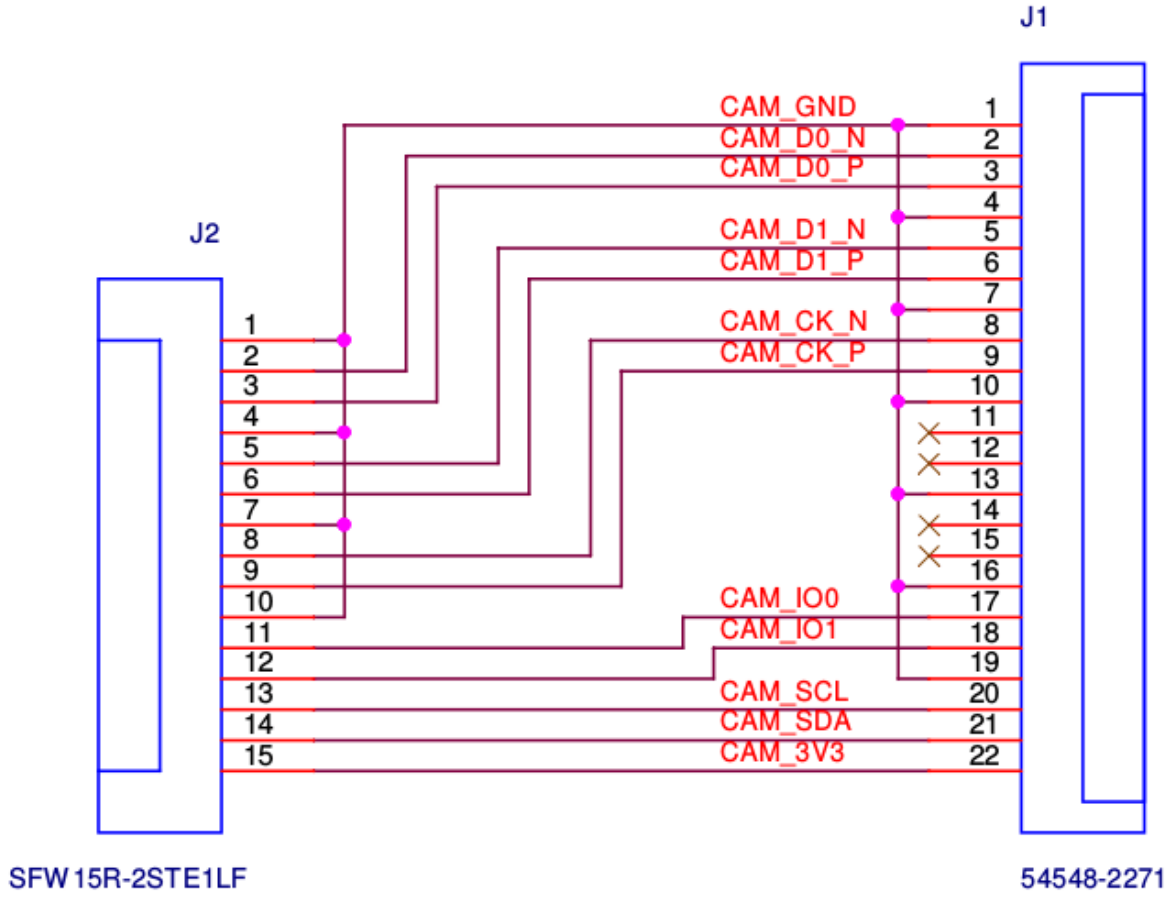


Рисунок 15 – Распиновка CSI разъём камеры

Таблица 3 – распиновка разъема

Pin	**Название**	**Описание**
1	Ground	Ground
2	CAM1_DN0	Data Lane 0
3	CAM1_DP0	
4	Ground	Ground
5	CAM1_DN1	Data Lane 1
6	CAM1_DP1	
7	Ground	Ground
8	CAM1_CN	MIPI Clock

Продолжение таблицы 2

9	CAM1_CP	
10	Ground	Ground
11	CAM_GPIO	
12	CAM_CLK	
13	SCL0	I²C Bus
14	SDA0	
15	+3.3 V	Power Supply

CAM1_CN и CAN1_CP: Эти контакты обеспечивают тактовый импульс для полос данных MIPI для первой камеры. Они подключаются к контактам MIPI Clock Positive (MCP) и MIPI Clock (MCN) ICC камеры. Эти тактовые сигналы обычно поступают от модуля камеры, генерируемого схемой MIPI.

CAM1_DN0 и CAN1_DP0: Это вывод данных MIPI Data Positive (MDPI) и MIPI Data negative (MDN) для полосы данных 0 камеры 1.

CAM1_DN1 и CAN1_DP1: Это вывод данных MIPI Data Positive (MDPI) и MIPI Data negative (MDN) для полосы данных 1 камеры 1.

SCL0 и SDA0: Меньшая последовательная шина, состоящая из контактов SCL и SDA, обеспечивает последовательную связь, которая позволяет пользователю управлять функциями камеры, такими как выбор разрешений. Эти контакты подключаются непосредственно к ведомому интерфейсу SCCB внутри IC камеры.

Вывод SCL обеспечивает стандартный входной тактовый сигнал последовательного интерфейса и стандартный последовательный интерфейс SDA для ввода/вывода данных.

4.5 Взаимодействие Raspberry Pi 3 с файловым хостингом Dropbox

В данном проекте файловый хостинг Dropbox выступает в качестве хранилища результатов работы программы. Raspberry Pi получает доступ к нему по сети Интернет через расширение Dropbox API, которое описано в документации от этой компании [8]. Предварительно нужно загрузить библиотеку dropbox в Python, в которой находятся методы, позволяющие загрузить, совместно использовать файлы и поддерживать другие операции с ними на прямую.

Чтобы получить доступ к Dropbox нужно сгенерировать API ключ и указать папку на Dropbox, куда нужно сохранить результат работы системы видеорегистрации движения объекта. На рисунке 16 представлен сгенерированный API ключ.

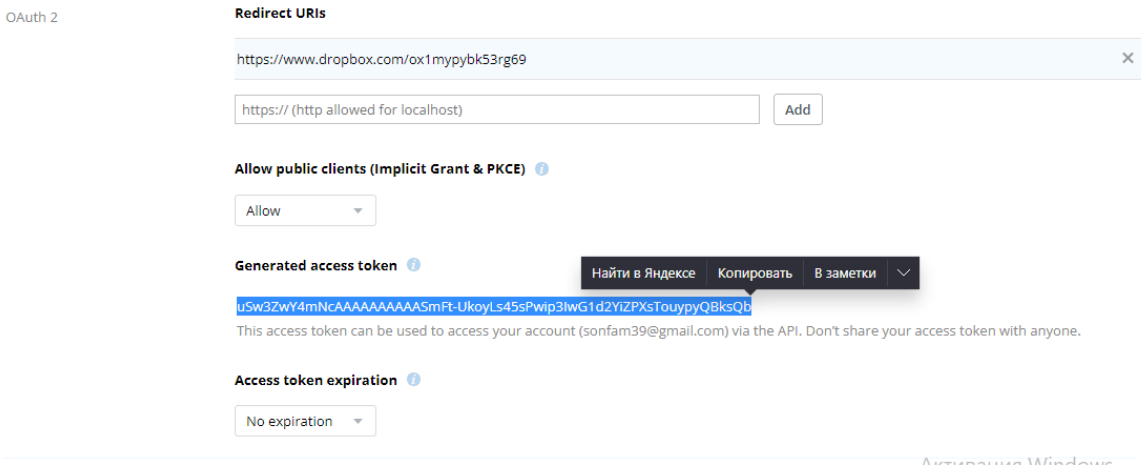


Рисунок 16 – Получение API ключа

Для передачи данных между сервером и клиентов в данном случае Dropbox и Raspberry Pi используется JSON объект (Приложение В), в котором хранятся следующие важные переменные (листинг 1).

```
    {
        "show_video": true,
        "use_dropbox": true,
        "dropbox_access_token":
"vpUh8Ke4aJcAAAAAAAAAAbaoCOTgYdp8DuDSYmccPuoC3ZDc4y3dZJ_m
LXGDQ27u",
```

```
        "dropbox_base_path":        "App/PI        Image
Search/Surveillance",
        "min_upload_seconds": 4.0,
        "min_motion_frames": 6,
        "camera_warmup_time": 2.5,
        "delta_thresh": 5,
        "resolution": [640, 480],
        "fps": 39,
        "min_area": 5000
    }
```

Листинг 1 – Содержание JSON объекта

- show_video: булевское значение, которое показывает, должен ли видеопоток с Raspberry Pi отображаться на экране.
- use_dropbox: булевское значение, которое показывает, следует ли использовать API Dropbox или нет.
- dropbox_access_token: ключ API Dropbox.
- dropbox_base_path: путь к папке, где хранятся загруженные изображения.
- min_upload_seconds: количество секунд ожидания между 2 загрузками. Этот параметр управляет частотой загрузки изображений.
- min_motion_frames: минимальное количество последовательных кадров, содержащих движение, перед тем, как изображение будет загружено в Dropbox.
- camera_warmup_time: количество секунд, в течение которых модуль камеры Raspberry Pi прогреться и откалиброваться.
- delta_thresh: модуль минимальной разницы между текущим кадром и усредненным кадром. Меньшие значения приведу к обнаружению большего количества движения, а большие значения к меньшему количеству обнаруженного движения.
- resolution: ширина и высота(разрешение) видеокадра с камеры.
- fps: частота кадров в секунду.
- min_area: минимальный размер области изображения (в пикселях), которая будет считаться за движением или не. Меньшие значения

приведут к большему количеству областей, отмеченных как движение, в то время как более высокие значения будут отмечать только боле крупные области как движение.

4.6 Сравнительный анализ и выбор алгоритма обнаружения движения объектов

При подборе решения нужно учесть тот факт, что разрабатываемая система развернута на Raspberry Pi 3 Model B, где вычислительные ресурсы ограничены по сравнению с полноценным ПК, поэтому выбранный метод должен быть простым, быстрым и эффективным. А также нужно не забывать о том, что производственный объект движется, и фон может измениться с течением времени. С учетом этих фактов был выбран распространенный метод вычитания фона, который был описан в книге о машинном зрении [9], но в нем существует множество разных способов реализации. Некоторые довольно простые, а некоторые очень сложные. Три основных метода:

- «An Improved Adaptive Background Mixture Model for Realtime Tracking with Shadow Detection» [10], разработанный KaewTraKulPong. Этот метод можно вызвать с помощью команды cv2.BackgroundSubtractorMOG.
- «Improved adaptive Gaussian Mixture Model for Background Subtraction», разработанный Zivkovic [11]. Его можно вызвать с помощью команды cv2.BackgroundSubtractorMOG2.
- «Visual Tracking of Human Visitors under Variable-Lighting Conditions for a Responsive Audio Art Installation» [12], разработанный Godbehere. Его можно вызвать с помощью команды cv2.createBackgroundSubtractorGMG.

Все эти методы работают на основе сегментации заднего фона от переднего фона, и они предоставляют инструменты, которые позволяют

различать реальное движение от тени или от небольших изменений освещения. Вышеперечисленные методы очень мощные, но у них есть большой недостаток: они дорогостоящие с точки зрения вычисления, что не подходит для Raspberry Pi.

В обычных задачах обнаружения движения объекта делают следующее предположение: фон нашего видеопотока статичный и неизменный на протяжении всего видеосъемки. Поэтому если существует возможность смоделировать фон, то с помощью вычитания текущего кадра от фона можно отследить объект. Результат вычитания – это движение в видеопотоке. На практике это предположение может не сработать. Солнечный свет из окна лаборатории с ГПС Denford или неисправность системы освещения в ней может привести к тому, что алгоритм не будет работать, как было задумано: алгоритм примет изменение фона за движения. Чтобы решить эту проблему нужно вычислить скользящее среднее значение прошлых N кадров (с учетом текущего) и принять “усредненный кадр” за фон. Это означает, что программа будет динамически подстраивается под фон. Затем нужно вычесть среднее значение из текущего кадра. Результат этой операции – дельта.

С помощью дельта программа находит область движения, сравнивает с константой min_area, если больше этого значения, то обводит ее в прямоугольник, если меньше, то пропускает. Сравнение с константой проводится с целью устранения ложных результатов в виде маленьких объектов, которые нас не интересуют. И в конце программа фиксирует кадр с обнаруженным движением. Блок-схема алгоритма представлена на рисунке 17.

Таким образом, был выбран метод обнаружения движения объекта, который достаточно простой, но в то же время эффективный для Raspberry Pi 3. А также в выбранном методе был учтен случай, когда фон может измениться из-за изменения условия освещения.

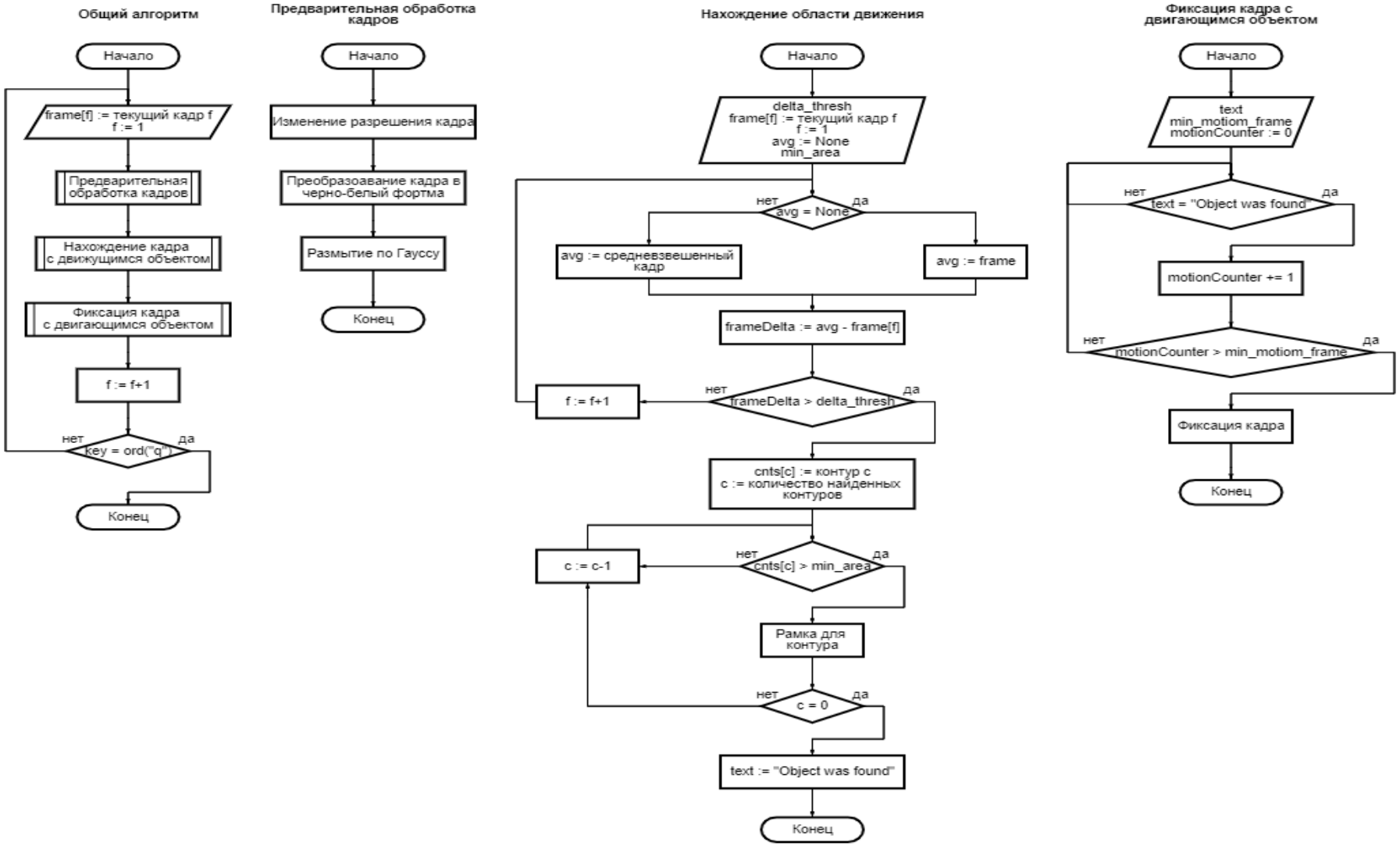


Рисунок 17 – Алгоритм обнаружение движения объекта

4.7 Сравнительный анализ и выбор алгоритма поиска регионов

В настоящее время существует множество алгоритмов поиска регионов, и традиционным считается метод, который использует пирамиду изображения в сочетании со скользящей рамкой. Пирамида изображений представляет собой многомасштабное представление изображения, то есть в нижней части пирамиды находится исходное изображение с исходными размерами, и чем выше поднимаемся по ней, тем меньше становится изображение. Она применяется для локализации объектов в разных масштабах. Как следует из названия, скользящая рамка представляет собой прямоугольник с фиксированным размером, который скользит слева направо и сверху вниз внутри изображения. Она позволяет точно определить местоположение объекта. В сочетании с пирамидами изображений скользящая рама может локализовать объекты в разных местах и в разных масштабах входного изображения. Но проблема этого алгоритма состоит в том, что он очень медленный и дорогостоящий в плане вычисления, поэтому этот метод не подходит в данной работе, где используется Raspberry Pi с ограниченной вычислительной мощностью.

Из вышеупомянутых соображений был выбран метод выборочного поиска, который был впервые представлен в статье [13], написанный Ụilings. Этот алгоритм идентифицирует перспективные регионы на изображении с помощью группировки смежных суперпикселей на основе некоторых критериев, а в подходе скользящего окна, где мы ищем объекты по всей картине и во всех его масштабах. Поэтому конечное число генерируемых предложений во много раз меньше, чем подход скользящего окна. Из-за этого метод выборочного поиска намного быстрее скользящей рамы с пирамидой изображений.

Выборочный поиск начинается с сегментации изображения на основе интенсивности пикселей с использованием графа. В данном методе используется

деление изображения, которое было разработано Felzenszwalb и Huttenlocher [14]. Он предназначен для предсказывания множество регионов, где возможно находятся объекты. Выборочный поиск использует полученные сегменты в качестве исходных данных и выполняет следующие шаги:

- добавляет ограничительные рамки к найденным сегментам;
- объединяет смежные сегменты на основе сходства, чтобы найти области изображения, которые могут содержать объект.

На каждой итерации формируются более крупные сегменты и добавляются в список перспективных регионов. В данном методе используется восходящий подход, то есть мы создаем из маленьких сегментов большие, которые и будут области, в которых возможно находится объект. Выборочный поиск объединяет сегменты иерархическим образом на основе пяти ключевых мер сходства:

- Сходство по цвету;
- Сходство по текстуре;
- Сходство по размеру;
- Сходство или совместимость формы;
- Мета сходство, то есть совокупность 4 вышеперечисленных.

Ниже представлена блок-схема алгоритма выборочного поиска (рисунок 18).

Рисунок 18 – Алгоритм поиска регионов

4.8 Сравнительный анализ и выбор алгоритма распознавания объекта

Основные алгоритмы системы распознавания движущихся объектов реализованы на Raspberry Pi, где вычислительная мощь ограничена, поэтому в данной работе невозможно обучение нейронной сети для распознавания движущихся объектов. Из данного умозаключения делаем выводы, что нужно использовать предобученную сеть. Выбор был сделан в пользу ResNet50, обученный на наборе данных ImageNet, так как ГПС Denford производит шахматные фигуры, а этот набор состоит из более 1000 классов объектов, среди которых присутствуют интересующие нас производственные объекты.

После применения алгоритма выборочного поиска, упомянутый в 4.7, мы получаем список регионов, в которых возможно находится нужный нам объект. Классификатор ResNet50 идентифицирует объекты в каждом предложенном регионе и на выходе выдает класс объекта, ограничивающая рамка и вероятность того, что объект внутри рамки принадлежит этому классу. В большинстве случаев результатом будут множества прямоугольников, содержащих интересующий нас объект. Для решения этой проблемы применяется метод Non-Maximum Suppression, который был упомянут в статье Felzenszwalb [15]. Он позволяет оставить только 1 наилучшую ограничивающую рамку и удалить остальные. На рисунке 19 представлена блок схема алгоритма распознавания алгоритма.

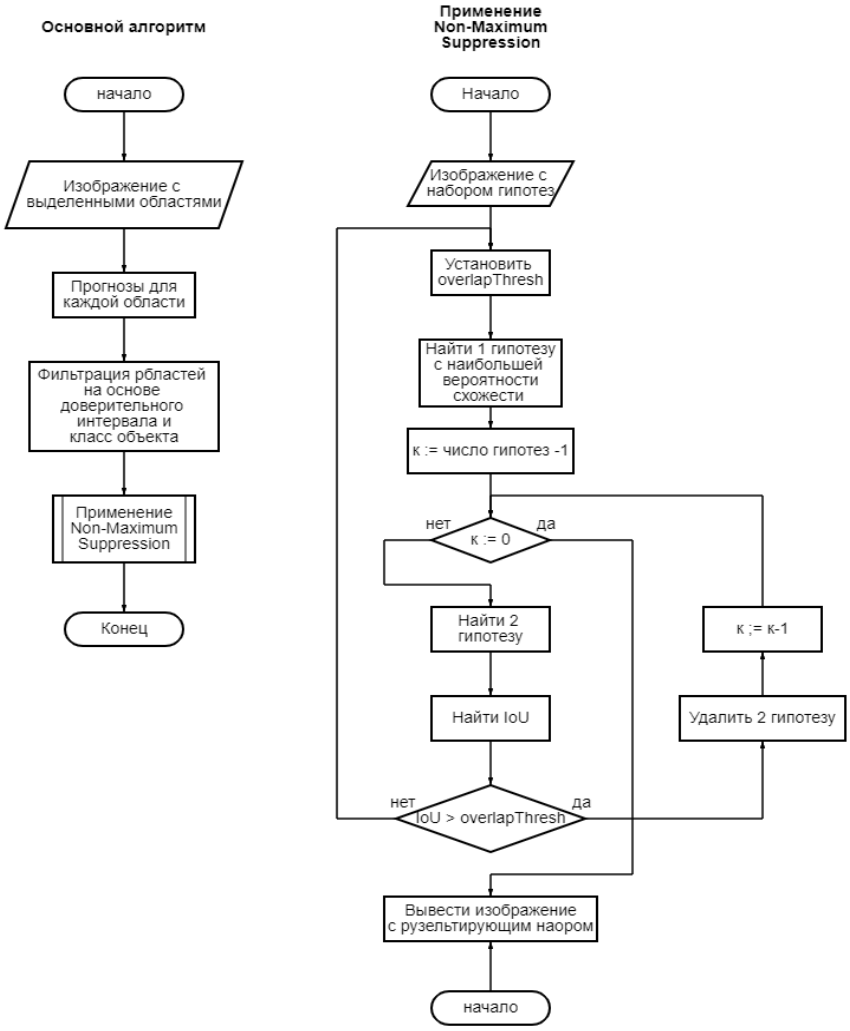


Рисунок 19 – Алгоритм распознавания объектов

5. Рабочее проектирование

Прежде чем приступать к программной части системы распознавания движущихся производственных объектов, необходимо разработать алгоритм подключения и взаимодействие Raspberry Pi и веб-приложение с помощью облачной системы. Весь код их взаимодействия описан в приложении Б.

Поэтапный алгоритм подключения Raspberry Pi к облачной платформе, который представлен на рисунке 20:

- оператор ГПС запускает ПО на Raspberry Pi;
- после начала работы программы одноплатный компьютер подключается к облачной платформе Yandex.Cloud с помощью логина и пароля, которые были внесены в программу при создании;
- после успешного соединения с MQTT брокером Raspberry Pi подписывается на необходимые топики;
- после подписки на нужные топики, в обработчик событий добавляются топик, по которому одноплатный компьютер ожидает получить информацию от облачной платформы;
- обработчик событий выполняет блоки кода в зависимости от полученных команд от облачной платформы Yandex.Cloud;
- ПО разработано так, что при нажатии клавиши «q» программа перестает работать и разрывается соединение с MQTT брокером облачной платформы.

Рисунок 20 – Алгоритм подключения Raspberry Pi к Yandex.Cloud

После подключения Raspberry Pi к MQTT-брокеру облачной платформы и подписки на определенные топики, сервер устройства готов принимать команды от Yandex.Cloud. Поэтапный алгоритм работы Raspberry Pi и веб-приложения, который представлен на рисунке 21:

- ожидание прихода сообщения в топик, на который подписан сервер устройства;

- в один из топиков, на который подписан одноплатный компьютер, приходит сообщение с выбранным классом объекта;
- полученный класс объекта в виде строки записывается в переменную;
- далее сравнивается с классами в ImageNet;
- если класс объекта присутствует в ImageNet, то включается программа распознавания движущихся объектов этого класса;
- результат программы, который содержит в себе фотографию с распознанным объектом и сообщение об его успешной идентификации, отправляется в один из топиков, на который подписан веб-приложение;
- веб-приложение отображает результат программы.

Рисунок 21 – Алгоритм работы Raspberry Pi и веб-приложения

5.1 Реализация подключения Raspberry Pi 3 к камере

Модуль камеры IMX219 подключается к Raspberry Pi с помощью шлейфа. После этого нужно включить разрешение на использование камеры с помощью командной строки sudo raspi-config (рисунок 22).

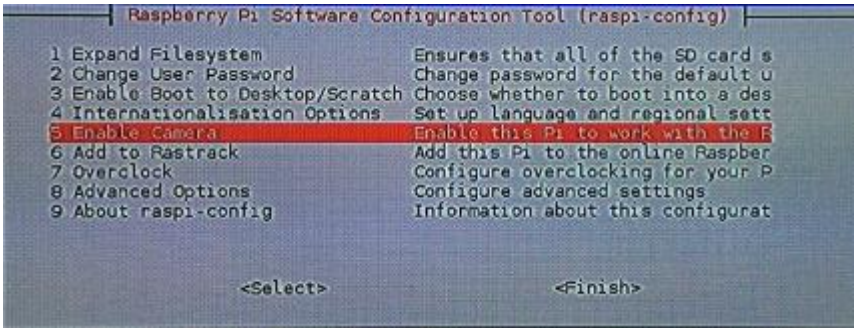


Рисунок 22 – Включение модуля камеры с помощью команды raspi-config

Взаимодействие между камерой и Python на одноплатном компьютере Raspberry Pi возможно при установке библиотеки picamera[array] (листинг 2) и было представлено в учебной литературе, посвященной компьютерному зрению [16]. Picamera предоставляет методы для взаимодействия с камерой, а подмодуль array позволяет получать изображения в виде массивов. Это важно, так как методы из OpenCV работают с изображениями в виде массивов NumPy.

В обычных условиях для получения видеопотока с камеры используется метод cv2.VideoCapture, но наша система развернута на Raspberry Pi, и для корректной работы этого метода нужно установить дополнительные драйвера и разобраться в сложном процессе настройки. В связи с этим в программе используется модуль picamera, который позволяет легко получить доступ к необработанному видеопотоку, вместо традиционного метода cv2.VideoCapture.

Процесс получения доступа к видеопотоку (листинг 2).

```
    camera = PiCamera()
    camera.resolution = tuple(conf["resolution"])
    camera.framerate = conf["fps"]
    rawCapture = PiRGBArray(camera,
size=tuple(conf["resolution"]))
    print("[INFO] warming up...")
    time.sleep(conf["camera_warmup_time"])
```

```
    avg = None
    lastUploaded = datetime.datetime.now()
```

Листинг 2 – Получение доступа к видеопотоку

Переменная camera позволяет нам взаимодействовать с камерой Raspberry Pi. Также мы настраиваем разрешения камеры и частоту кадров. В файле conf в аргументе resolution и fps можно настраивать разрешение и частоту кадров вручную. Это было сделано по следующим соображениям:

- При желании изменить разрешение, не нужно по всему коду искать этот параметр и изменять его. Этот процесс может привести к дальнейшим ошибкам из-за человеческого фактора;
- Иногда появляется такая ситуация, что в командной строке слишком много аргументов, и такой метод позволяет лучше понять, что представляет собой каждый аргумент;

Далее происходит инициализация массива. Метод PiRGBArray использовали для получения упорядоченного трехмерного RGB-массива numpy, т.е. изображение с камеры. Также модулю камеры Raspberry Pi нужно прогреться в течение нескольких секунд. Этот процесс гарантирует, что датчикам предоставили достаточно времени для калибровки. В конце инициализированы средний кадр заднего фона и бухгалтерские переменные (дата, время).

5.2 Реализация подключения Raspberry Pi 3 к файловому хостингу Dropbox

Raspberry Pi получает доступ к Dropbox через расширение Dropbox API. Для интеграции с ним нужно настроить аккаунт, получить API ключ и создать папку, где будет храниться результат программы.

Подключение к Dropbox (листинг 3).

```
    if conf["use_dropbox"]:
        client                                          =
dropbox.Dropbox(conf["dropbox_access_token"])
```

```
print("[SUCCESS] dropbox account linked")
```

Листинг 3 – Получение доступа к видеопотоку

Идет проверка, чтобы узнать, следует ли использовать Dropbox или нет. Аргумент use_dropbox имеет значение true/false. Он находится в файле conf. При положительном результате авторизует программу с помощью ключа API, и система подключается к Dropbox.

Затем идет загрузка результата программы на Dropbox (листинг 4).

```
                        if conf["use_dropbox"]:
                            t = TempImage()
                            cv2.imwrite(t.path, clone)
                            print("[UPLOAD] {}".format(ts))
                            path                            =
"/{base_path}/{timestamp}.jpg".format(
                        base_path=conf["dropbox_base_path"],
timestamp=ts)
                            client.files_upload(open(t.path,
"rb").read(), path)
                            t.cleanup()
```

Листинг 4 – Отправка результата программы в хранилище

Программа записывает результат во временную папку хранения TempImage (Приложение Г), затем отправляет в файловый хостинг через Dropbox API, и наконец очищает эту папку для следующей фотографии.

5.3 Реализация основных алгоритмов системы распознавания движущихся производственных объектов

Были разработаны основные алгоритмы, позволяющие обнаруживать и идентифицировать движущиеся производственные объекты. Полный листинг программы представлен в приложении А.

В начале импортируем необходимые для работы библиотеки (листинг 5).

```
    from tensorflow.keras.applications import ResNet50
    from  tensorflow.keras.applications.resnet50  import
preprocess_input
```

```
    from tensorflow.keras.applications import
imagenet_utils
    from tensorflow.keras.preprocessing.image import
img_to_array
    from imutils.object_detection import
non_max_suppression
    from pyimagesearch.tempimage import TempImage
    from picamera.array import PiRGBArray
    from picamera import PiCamera
    import argparse
    import warnings
    import datetime
    import dropbox
    import imutils
    import json
    import time
    import cv2
    import numpy as np
```

Листинг 5 – Импорт необходимых библиотек

Программа начинается с нескольких импортов. В частности, предварительно обученный классификатор ResNet50, метод Non-Maximum Suppression для однозначного определения объекта и OpenCV, который содержит функции для обработки изображений. Помимо стандартных библиотек для работы с компьютерным зрением импортируем класс TempImage, позволяющий хранить полученный результат перед его отправкой на Dropbox. PiRGBArray даёт прямой доступ к потоку камеры и избежать дорогостоящего сжатия в формате JPEG, который в дальнейшем всё равно нужно декодировать в формат OpenCV. Также нужно уделить внимание тому, что OpenCV представляет изображение в виде массивов NumPy, для работы с которыми был импортирован соответствующий пакет, в порядке BGR, а не RGB. Методы из библиотеки picamera позволяют получить доступ к необработанному видеопотоку камеры Raspberry Pi. Импорт Dropbox дает возможность взаимодействовать с файловым хостингом напрямую через методы, хранящие в этой библиотеке. JSON объекты, для работы с которой были изучены их особенности в электронном ресурсе [17], помогают в процессе обмена информации между сервером устройства и файловым хостингом или облачным

сервисом. Пакет imutils содержит в себе набор удобных функций для упрощения основных задач обработки изображений.

Следующим шагом является определение входных данных для программы (листинг 6).

```
    ap = argparse.ArgumentParser()
    ap.add_argument("-c", "--conf", required=True,
        help="path to the JSON configuration file")
    ap.add_argument("-m",      "--method",      type=str,
default="fast",
        choices=["fast", "quality"],
        help="selective search method")
    ap.add_argument("-f",      "--filter",      type=str,
default=None,
        help="comma separated list of ImageNet labels to
filter on")
    ap.add_argument("-p",      "--prob",      type=float,
default=0.9,
        help="minimum   probability   to   consider   a
classification/detection")
    args = vars(ap.parse_args())
```

Листинг 6 – Входные данные

- --conf является переключателем для регулировки параметров камеры, а также путем к файлу JSON;
- --method представляет собой режим выборочного поиска «быстрый» либо «медленный»;
- --filter в нем хранится классы объектов ImageNet, разделенные запятыми, которые нас интересуют.
- --prob это минимальный порог вероятности для рассмотрения классификации.

Далее идет описание метода выборочного поиска (листинг 7).

```
    def selective_search(image, method="fast"):
        ss                                              =
cv2.ximgproc.segmentation.createSelectiveSearchSegmentati
on()
        ss.setBaseImage(image)
        if method == "fast":
            ss.switchToSelectiveSearchFast()
```

```
        else:
            ss.switchToSelectiveSearchQuality()
        rects = ss.process()
        return rects
```

Листинг 7 – Создание метода выборочного поиска

Функция selective_search принимает на вход изображение и алгоритмические методы «быстро» или «медленно». Далее происходит инициализация выборочного поиска с помощью входного фото. Затем устанавливается режим работы метода. В основном подходит быстрый метод, потому что в данном проекте система работает с движущимися объектами и требует высокой производимости программы. Однако в зависимости от поставленных задач можно пожертвовать скоростью для достижения лучших результат. В конце выполняется выборочный поиск и метод возвращает список с перспективными регионами. Когда вызывается функция selective_search, получаем ограничивающие рамки, которые показывают, где возможно находится объект.

Далее загружаем предварительно обученный классификатор изображений ResNet (листинг 8).

```
    model = ResNet50(weights="imagenet")
```

Листинг 8 – Классификатор ResNet

Здесь инициализируется ResNet, предварительно обученный на ImageNet.

Следующим шагом будет реализация алгоритма обнаружение движения объектов (листинг 9).

```
    avg = None
    lastUploaded = datetime.datetime.now()
    motionCounter = 0
    for   f   in   camera.capture_continuous(rawCapture,
format="bgr", use_video_port=True):
        frame = f.array
        timestamp = datetime.datetime.now()
        text = "No objects"
        frame = imutils.resize(frame, width=500)
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        gray = cv2.GaussianBlur(gray, (21, 21), 0)
        if avg is None:
            print("[INFO] starting background model...")
```

```
            avg = gray.copy().astype("float")
            rawCapture.truncate(0)
            continue
        cv2.accumulateWeighted(gray, avg, 0.5)
        frameDelta          =          cv2.absdiff(gray,
cv2.convertScaleAbs(avg))
    thresh          =          cv2.threshold(frameDelta,
conf["delta_thresh"], 255,
            cv2.THRESH_BINARY)
        thresh = cv2.dilate(thresh, None, iterations=2)
        cnts      =      cv2.findContours(thresh.copy(),
cv2.RETR_EXTERNAL,
            cv2.CHAIN_APPROX_SIMPLE)
        cnts = imutils.grab_contours(cnts)
        for c in cnts:
            if cv2.contourArea(c) < conf["min_area"]:
                continue
            text = "Object was found"
        ts = timestamp.strftime("%A %d %B %Y %I:%M:%S%p")
        cv2.putText(frame,        "Object        Status:
{}".format(text), (10, 20),
            cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 255),
2)
        cv2.putText(frame, ts, (10, frame.shape[0] - 10),
cv2.FONT_HERSHEY_SIMPLEX,
            0.35, (0, 0, 255), 1)
    if text == "Object was found":
            if  (timestamp  -  lastUploaded).seconds  >=
conf["min_upload_seconds"]:
                motionCounter += 1
                if           motionCounter           >=
conf["min_motion_frames"]:
```

Листинг 9 – Алгоритм получении фото движущихся объектов

Начинается алгоритм с инициализации среднего фонового кадра, на котором помимо фона еще будут отображаться время и день. Предварительно программа обрабатывает каждый кадр из видеопотока, изменяя размер до 500 пикселей, затем преобразует его в черно-белый формат и применяет размытие по Гауссу. Изменение размера и преобразование изображения в черно-белый формат необходимы для того, чтобы программы обрабатывала кадры быстрее. Размытие по Гауссу удаляет высокочастотные шумы и позволяет программе сосредоточится на нужных объектах изображения. Далее идет проверка,

инициализировано ли усредненный кадр или нет. Если нет, то текущий кадр принимаем за усредненный. Затем программа берет средневзвешенное значение предыдущих кадров вместе с текущим кадром. Это означает, что алгоритм может динамически подстраиваться под фон. Основываясь на средневзвешенном значении кадров, мы вычитаем его от текущего кадра и получаем дельта. Основываясь на пороговое значение этой переменной, программа находит области кадра с существенным отличием от фоновой модели. Так находится «движение» в видеопотоке. Для нахождения таких областей применяется обнаружение контуров. Затем перебирается каждый из них по отдельности и сравниваем с пороговым значением. Если регионы достаточно большие, то это воспринимается за нахождение движения в этом кадре. Далее алгоритм вычисляет ограничивающую рамку контура и рисует ее.

Следующим шагом является реализация алгоритма выборочного поиска (листинг 10).

```
        if text == "Object was found":
            if (timestamp - lastUploaded).seconds >=
conf["min_upload_seconds"]:
                motionCounter += 1
                if motionCounter >=
conf["min_motion_frames"]:
            (H, W) = frame.shape[:2]
                    rects = selective_search(frame,
method=args["method"])
                    proposals = []
                    boxes = []
                    for (x, y, w, h) in rects:
                        if w / float(W) < 0.1 or h /
float(H) < 0.1:
                            continue
                        roi = frame[y:y + h, x:x + w]
                        roi = cv2.cvtColor(roi,
cv2.COLOR_BGR2RGB)
                        roi = cv2.resize(roi, (224, 224))
                        roi = img_to_array(roi)
                        roi = preprocess_input(roi)
                        proposals.append(roi)
                        boxes.append((x, y, w, h))
```

Листинг 10 – Реализация выборочного поиска

В начале алгоритм производит проверку на обнаружение движения в текущем кадре. Если это так, то программа проверяет, прошло ли достаточно времени между настоящим моментом и предыдущей загрузкой в Dropbox. Если это верно, то счетчик движения увеличивается на 1. При достижении достаточного движения в счетчике применяется выборочный поиск в нужном нам режиме. Результатом является список перспективных регионов, хранящихся в переменной rects. Далее инициализированы 2 переменные. В первом хранится список области с большой вероятностью содержания объекта, а во втором находится список координат их ограничивающих рамок. Во всех регионах происходят фильтрация полей, где вероятно не содержатся объекты, предварительная обработка и добавляем их в вышеуказанные переменные.

Финальным шагом является реализация алгоритма распознавания объекта (листинг 11).

```
        proposals = np.array(proposals)
        preds = model.predict(proposals)
    preds    =    imagenet_utils.decode_predictions(preds,
top=1)
        labels = {}
        for (i, p) in enumerate(preds):
            (imagenetID, label, prob) = p[0]
            if labelFilters is not None and label not in
labelFilters:
                continue
            if prob >= args["prob"]:
                (x, y, w, h) = boxes[i]
        box = (x, y, x + w, y + h)
        L = labels.get(label, [])
        L.append((box, prob))
        labels[label] = L
        for label in labels.keys():
            clone = image.copy()
            boxes    =    np.array([p[0]    for    p    in
labels[label]])
            proba    =    np.array([p[1]    for    p    in
labels[label]])
            boxes = non_max_suppression(boxes, proba)
            for (startX, startY, endX, endY) in boxes:
```

```
                cv2.rectangle(clone,  (startX,  startY),
(endX, endY), (0, 255, 0), 2)
                y = startY - 10 if startY - 10 > 10 else
startY + 10
                cv2.putText(clone,  label,  (startX,  y),
cv2.FONT_HERSHEY_SIMPLEX, 0.45, (0, 255, 0), 2)
```

Листинг 11 – Реализация алгоритма распознавания объектов

Для начала переменная proposals в виде списка перспективных областей преобразуется в массив NumPy. С помощью предварительно обученного классификатора делаются прогнозы для каждого регионы из proposals, то есть показывает, насколько объект внутри данной области похож с классом объекта, который был выбран ранее. Далее происходит фильтрация результатов на основе доверительного порога и класса объекта. В переменную labels записываются отфильтрованные результаты в виде словаря, состоящего из классов объекта, ограничивающие рамки вероятность схожести объектов. И В конце применяется метод Non-Maximum Suppression для устранения случаев, когда несколько ограничивающиеся рамки показывают 1 и тот же объекта.

6. Апробирование

На веб-приложении оператор ГПС выбирает класс объекта, который нужно распознать (рисунок 22) и нажимает кнопку «Отправить» для того, чтобы дать команду системе распознавания начать поиск нужного объекта.

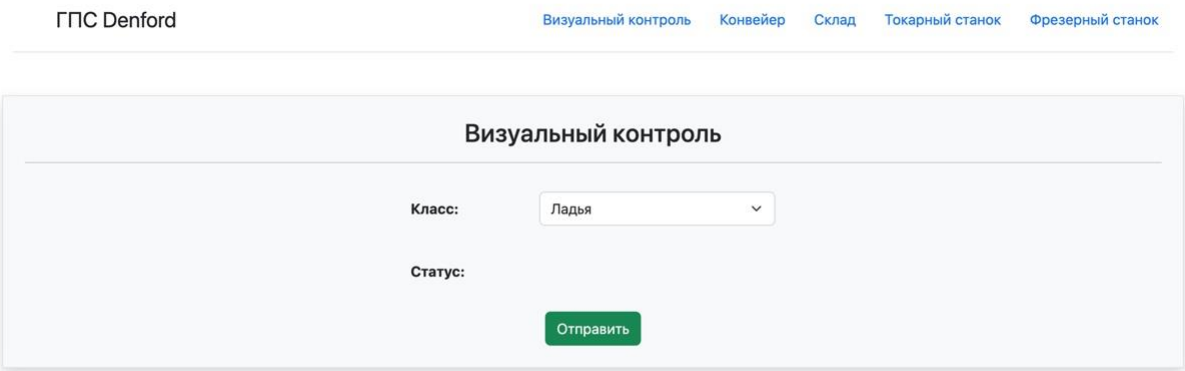


Рисунок 22 – Выбор класс объекта на веб-приложении

Raspberry Pi принял сообщение и запустил нужную подпрограмму для распознавания движущихся объектов выбранного класса (рисунок 23).

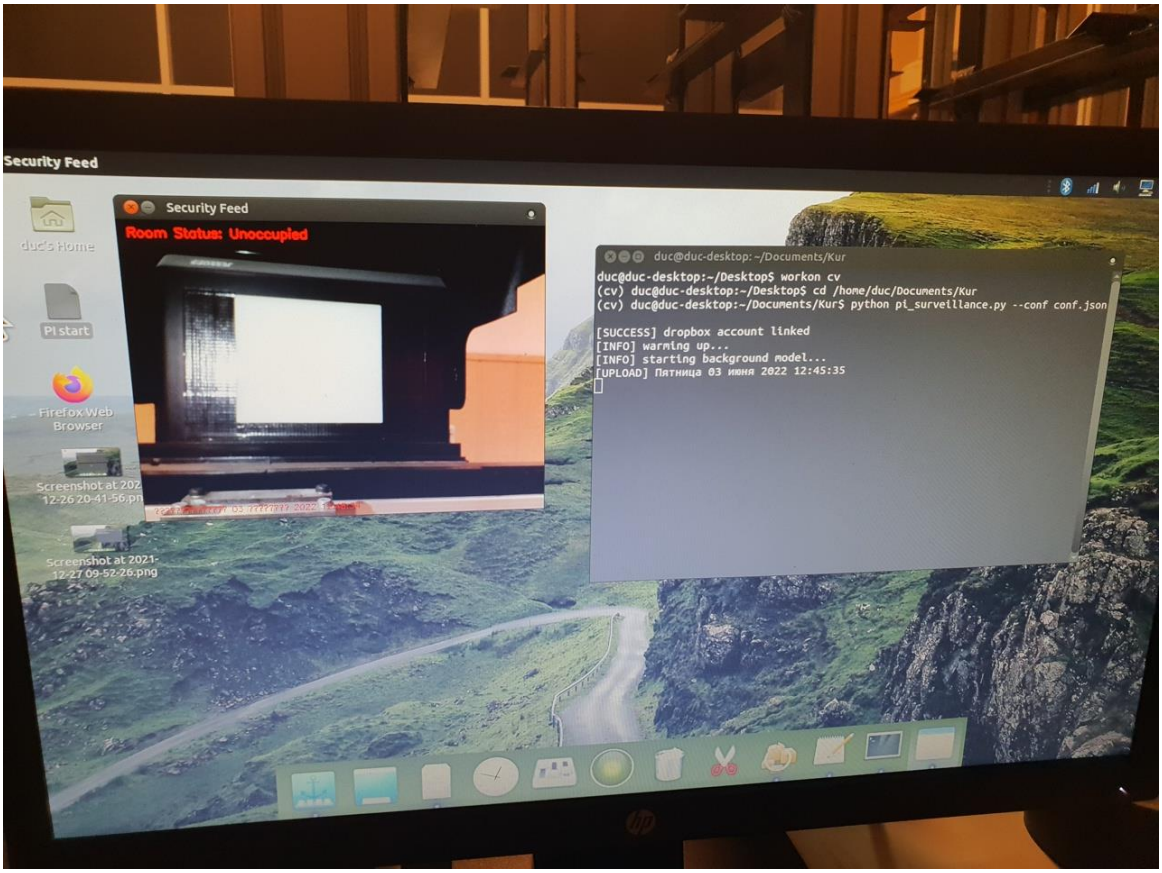


Рисунок 23 – Запуск подпрограммы распознавания

Программа распознала объекты выбранного класса и отправила результат в файловый хостинг Dropbox (рисунок 24).

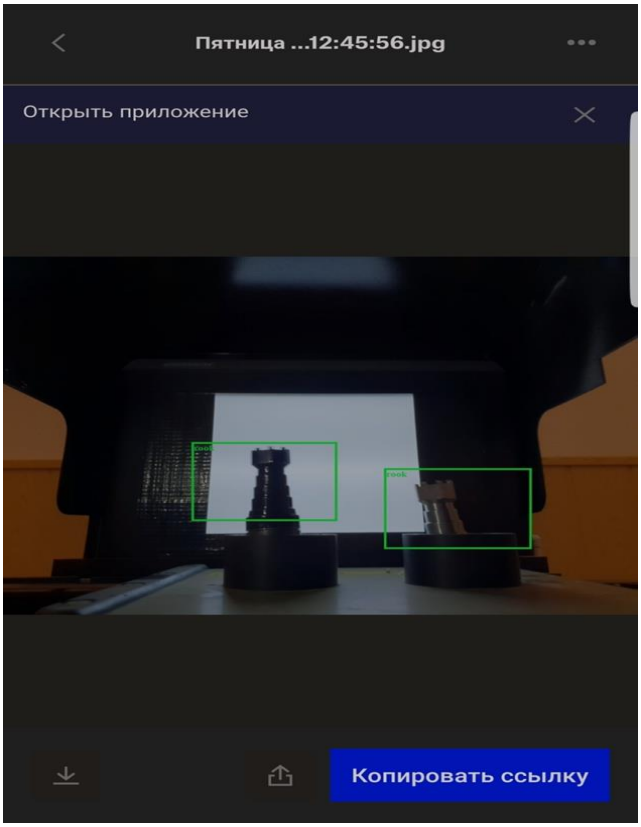


Рисунок 24 – Результат программы в файловом хранилище Dropbox

Также результат работы системы распознавания движущихся производственных объектов отправляется на веб-приложение (рисунок 25).

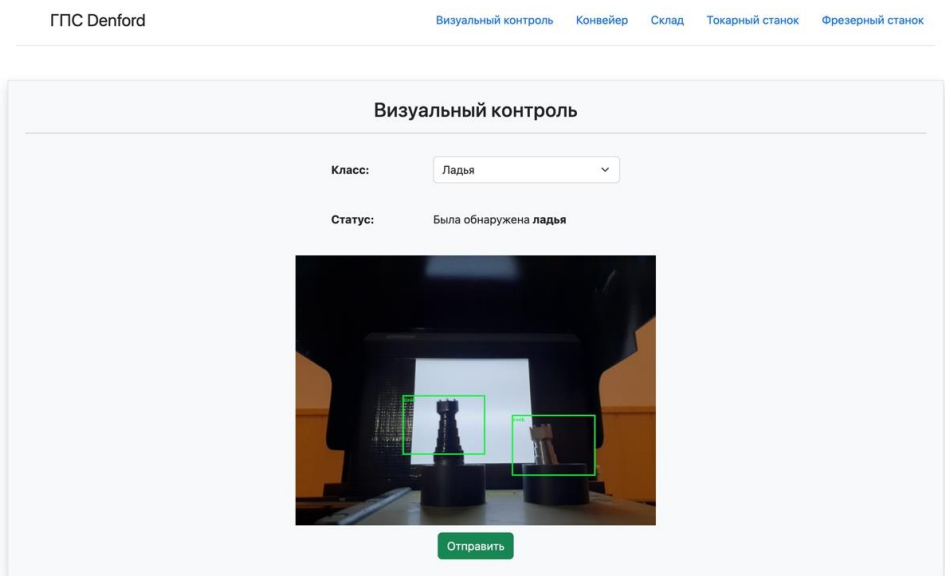


Рисунок 25 – Отображение результата программы на веб-приложении

ЗАКЛЮЧЕНИЕ

В ходе проделанной работы были получены следующие результаты:

1) Произведен анализ существующей системы распознавания, который показал, что она имеет ряд недостатков.
2) Были сформированы требования к разрабатываемой программной части системы распознавания движущихся производственных объектов. Один из важнейших – возможность автономной работы при подключении к Web-приложению через облачную платформу Yandex Cloud.
3) Были разработаны: функциональная модель распознавания движущихся объектов в автономном режиме, концептуальная модель взаимодействия web-приложения, облачной платформы, Raspberry Pi и модуля камеры.
4) Были реализованы: основные алгоритмы системы распознавания движущихся производственных объектов, подключение Raspberry Pi к камере, подключение Raspberry Pi к файловому хостингу Dropbox, к Yandex.Cloud.

Разработанная система распознавания движущихся производственных объектов в облачной системе управления реализует концепцию промышленного интернета вещей в рамках рассматриваемого складского модуля ГПС Denford.

Эта система позволяет оператору взаимодействовать с камерой через Web-приложение: команды на распознавания движущихся объектов определенного класса передаются Raspberry Pi по сети Интернет через протокол MQTT от облачной платформы Yandex.Cloud.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1) Соколов Д. Индустрия 4.0: Big Data, цифровизация и рост экономики. [Электронный ресурс] // Habr [сайт]. [2020]. URL: https://habr.com/ru/post/507822/ (дата обращения 29.04.2022)

2) ГОСТ 16504-81. Система государственных испытаний продукции. Испытания и контроль качества продукции. Основные термины и определения. – Взамен ГОСТ 16504-74; Введ. 1982 – 01 – 01. – Москва: Москва Стандартинформ, 2011. – 25 с.

3) Овсянников М. В., Буханов С. А., Подкопаев С. А. Организация индустриального интернета на основе SAP HANA Cloud Platform: Учебн. пособие. – М.: издательство МГТУ им. Н. Э. Баумана, 2019. – 195 с.

4) ГОСТ 34.602-89. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы. – Москва: Москва Стандартинформ, 2009. – 12 с.

5) Getting started with Raspberry Pi [Электронный ресурс] // Raspberrypi [сайт]. [2021] URL: https://projects.raspberrypi.org/en/projects/raspberry-pi-getting-started (дата обращения 29.04.2022)

6) Puler T. Hands-On Internet of Things with MQTT. – Birmingham: Packt, 2019. – 350 p.

7) Пошаговые инструкции Yandex.Cloud – Документация. [Электронный ресурс] // Cloud.Yandex [сайт]. [2020] URL: https://cloud.yandex.ru/docs/iot-core/operations/publish (дата обращения 29.04.2022)

8) Dropbox for Python Documentation – Документация. [Электронный ресурс] // Dropbox [сайт]. [2021] URL: https://dropbox-sdk-python.readthedocs.io/en/latest/ (дата обращения 29.04.2022)

9) Визильтер Ю. В., Желтов С. Ю., Бондаренко А. В., Ососков М.В. Моржин А. В. Обработка и анализ изображений в задачах машинного зрения. — М.: Физматкнига, 2010. — 689 с.

10) KaewTraKulPong P., Bowden R. An Improved Adaptive Background Mixture Model for Realtime Tracking with Shadow Detection. – 2nd European Workshop, 2001 – 5 p.

11) Zoran Z. Improved Adaptive Gaussian Mixture Model for Background Subtraction. – ICPR, 2004 – 4 p.

12) Godbehere A. B., Akihiro M., Goldberg K. Visual Tracking of Human Visitors under Variable-Lighting Conditions for a Responsive Audio Art Installation. – American Control Conference (ACC), 2012 – 5 p.

13) Uijlings J.R.R., Sande K.E.A., Gevers T., Smeulders A.W.M. Selective Search for Object Recognition. – IJCV, 2012 – 14 p.

14) Felzenszwalb P.F., Huttenlocher D.P. Efficient Graph-Based Image Segmentation. – The Netherlands: IJCV, 2003 – 13 p.

15) Felzenszwalb P.F., Girshick R.B., McAllester D., Ramanan D. Object Detection with Discriminatively Trained Part Based Models. – IEEE, 2009 – 20 p.

16) Joseph H. OpenCV Computer Vision with Python. – Birmingham: Packt, 2013 - 105 p.

17) Работа с файлами в формате JSON [Электронный ресурс] // JSON [сайт]. [2021]. URL: https://pyneng.readthedocs.io/ru/latest/book/17_serialization/json.html (дата обращения 29.04.2022)

Приложение А

Исходный код основных алгоритмов в системе распознавания

```
from pyimagesearch.tempimage import TempImage
from picamera.array import PiRGBArray
from picamera import PiCamera
import argparse
import warnings
import datetime
import dropbox
import imutils
import json
import time
import cv2

ap = argparse.ArgumentParser()
ap.add_argument("-c", "--conf", required=True,
     help="path to the JSON configuration file")
args = vars(ap.parse_args())
warnings.filterwarnings("ignore")
conf = json.load(open(args["conf"]))
client = None
if conf["use_dropbox"]:
     client = dropbox.Dropbox(conf["dropbox_access_token"])
     print("[SUCCESS] dropbox account linked")
camera = PiCamera()
camera.resolution = tuple(conf["resolution"])
camera.framerate = conf["fps"]
rawCapture = PiRGBArray(camera, size=tuple(conf["resolution"]))
print("[INFO] warming up...")
time.sleep(conf["camera_warmup_time"])
avg = None
lastUploaded = datetime.datetime.now()
motionCounter = 0
for   f   in   camera.capture_continuous(rawCapture,   format="bgr",
use_video_port=True):
     frame = f.array
     timestamp = datetime.datetime.now()
     text = "Unoccupied"
     frame = imutils.resize(frame, width=500)
     gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
     gray = cv2.GaussianBlur(gray, (21, 21), 0)
     if avg is None:
          print("[INFO] starting background model...")
          avg = gray.copy().astype("float")
          rawCapture.truncate(0)
          continue
     cv2.accumulateWeighted(gray, avg, 0.5)
     frameDelta = cv2.absdiff(gray, cv2.convertScaleAbs(avg))
```

```
    thresh = cv2.threshold(frameDelta, conf["delta_thresh"], 255,
        cv2.THRESH_BINARY)[1]
    thresh = cv2.dilate(thresh, None, iterations=2)
    cnts = cv2.findContours(thresh.copy(), cv2.RETR_EXTERNAL,
        cv2.CHAIN_APPROX_SIMPLE)
    cnts = imutils.grab_contours(cnts)
    for c in cnts:
        if cv2.contourArea(c) < conf["min_area"]:
            continue
        (x, y, w, h) = cv2.boundingRect(c)
        cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0),
2)
        text = "Occupied"
    ts = timestamp.strftime("%A %d %B %Y %I:%M:%S%p")
    cv2.putText(frame, "Room Status: {}".format(text), (10, 20),
        cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 255), 2)
    cv2.putText(frame,   ts,   (10,   frame.shape[0]   -   10),
cv2.FONT_HERSHEY_SIMPLEX,
        0.35, (0, 0, 255), 1)
    if text == "Occupied":
        if     (timestamp     -     lastUploaded).seconds     >=
conf["min_upload_seconds"]:
            motionCounter += 1
            if motionCounter >= conf["min_motion_frames"]:
                if conf["use_dropbox"]:
                    t = TempImage()
                    cv2.imwrite(t.path, frame)
                    print("[UPLOAD] {}".format(ts))
                    path                                    =
"/{base_path}/{timestamp}.jpg".format(
                        base_path=conf["dropbox_base_path"],
timestamp=ts)
                    client.files_upload(open(t.path,
"rb").read(), path)
                    t.cleanup()
                lastUploaded = timestamp
                motionCounter = 0
    else:
        motionCounter = 0
    if conf["show_video"]:
        cv2.imshow("Security Feed", frame)
        key = cv2.waitKey(1) & 0xFF
        if key == ord("q"):
            break
    rawCapture.truncate(0)
```

Приложение Б

Исходный код взаимодействия Raspberry Pi с

```
from os import stat_result
import requests
from django.http import HttpResponseRedirect
from django.shortcuts import render
from django.http import JsonResponse
import paho.mqtt.client as mqtt
import ssl
import time
from subprocess import Popen
import sys
type = None
conv = mqtt.Client(client_id="arefkga74np0lpbahno4")
conv.tls_set(ca_certs="/Users/dekr/Desktop/rootCA.crt",
                certfile="/Users/dekr/Desktop/cert.pem",
                keyfile="/Users/dekr/Desktop/key.pem",
                cert_reqs=ssl.CERT_REQUIRED,
                tls_version=ssl.PROTOCOL_TLSv1_2)
conv.connect("mqtt.cloud.yandex.net", port=8883, keepalive=5)
client.subscribe('$devices/arefkga74np0lpbahno4/state/class')
if msg.topic == '$devices/arefkga74np0lpbahno4/state/class':
    global type
    type = msg.payload.decode()
if type == "king":
    Popen([sys.executable,  'region_proposal_detection.py  --filter
king'])
else if type == "queen":
    Popen([sys.executable,  'region_proposal_detection.py  --filter
queen'])
else if type == "rook":
    Popen([sys.executable,  'region_proposal_detection.py  --filter
rook'])
else if type == "bishop":
    Popen([sys.executable,  'region_proposal_detection.py  --filter
bishop'])
else if type == "knight":
    Popen([sys.executable,  'region_proposal_detection.py  --filter
knight'])
else if type == "pawn":
    Popen([sys.executable,  'region_proposal_detection.py  --filter
pawn'])
config_json      =      'https://storage.yandexcloud.net/fms-file-
bucket/config.json'
config_info = requests.get(config_json).json()
context = {
        "статус": "была обнаружена ",type,
        " ": frame,
```

```
    }
client.publish('$devices/arefkga74np0lpbahno4/state/class',
context)
if key == ord("q"):
    brea
```

Приложение В

Файл conf.json


```
{
     "show_video": true,
     "use_dropbox": true,
     "dropbox_access_token":
"vpUh8Ke4aJcAAAAAAAAAAbaoCOTgYdp8DuDSYmccPuoC3ZDc4y3dZJ_mLXGDQ27u"
,
     "dropbox_base_path": "App/PI Image Search/Surveillance",
    "min_upload_seconds": 4.0,
     "min_motion_frames": 6,
     "camera_warmup_time": 2.5,
     "delta_thresh": 5,
     "resolution": [640, 480],
     "fps": 39,
     "min_area": 5000
}
```

Приложение Г

Класс TempImg

```
import uuid
import os
class TempImage:
     def __init__(self, basePath="./", ext=".jpg"):
          self.path                                                    =
"{base_path}/{rand}{ext}".format(base_path=basePath,
               rand=str(uuid.uuid4()), ext=ext)
     def cleanup(self):
          os.remove(self.path)
```

Приложение Д

Графическая часть

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

Выпускная квалификационная работа на тему:

«Система распознавания движущихся производственных объектов в облачной системе управления»

Выполнил студент РК9И-85Б:

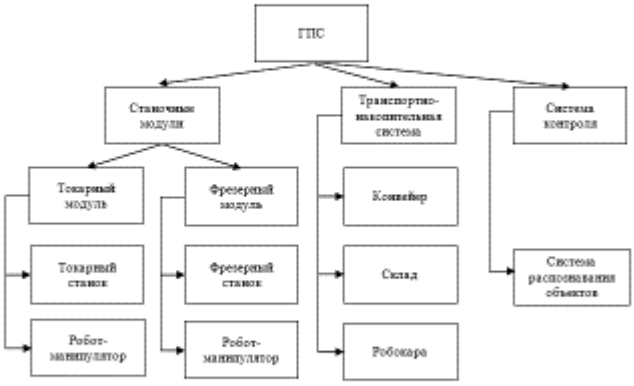
Фам Т.Ш.

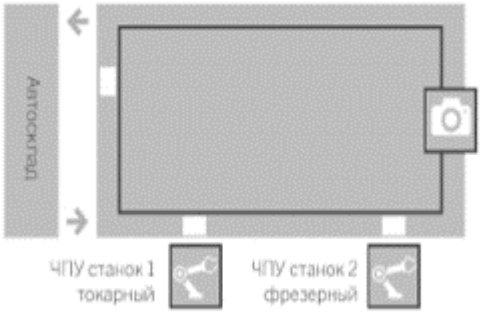
Руководитель:

Буханов С.А.

Москва, 2022

Анализ существующей системы





2

Цели и задачи

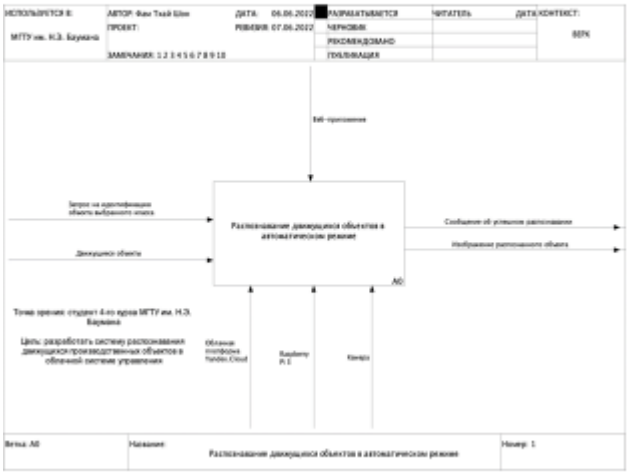
Цель работы: разработать систему распознавания движущихся производственных объектов в облачной системе управления.

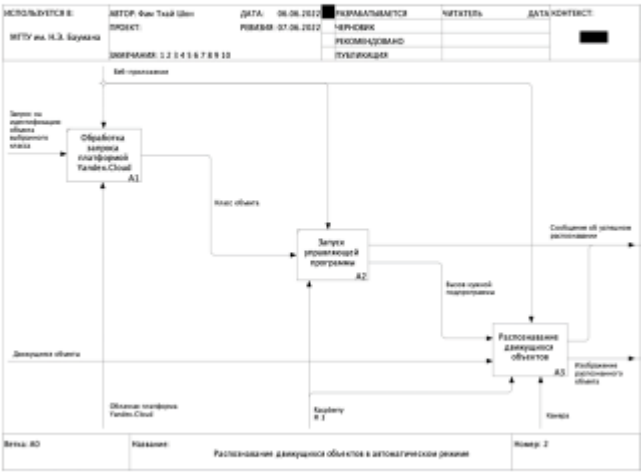
В ходе проекта были сформулированы следующие задачи:

- проанализировать существующую систему распознавания и выявить ее недостатки;
- на основе анализа полученных данных сформулировать требования, предъявляемые к разрабатываемой системе управления;
- разработать функциональную модель распознавания движущихся объектов;
- разработать алгоритмы основных функций программ;
- разработать программу для взаимодействия Raspberry Pi, файлового хостинга и веб-приложения.

3

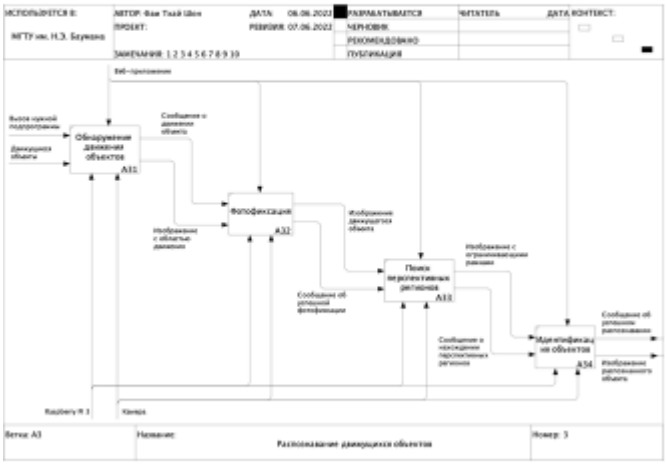
Функциональная модель

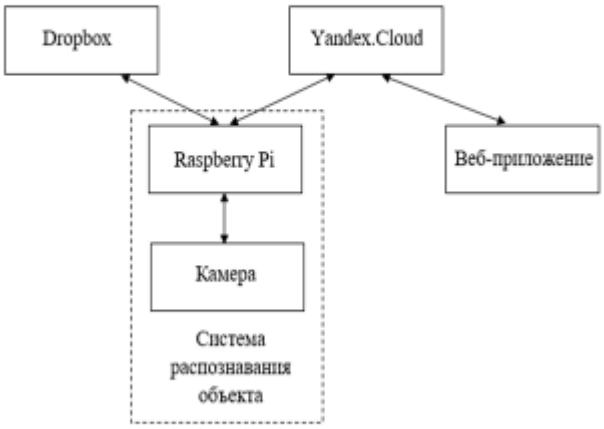




4

Функциональная модель и архитектура разрабатываемой системы





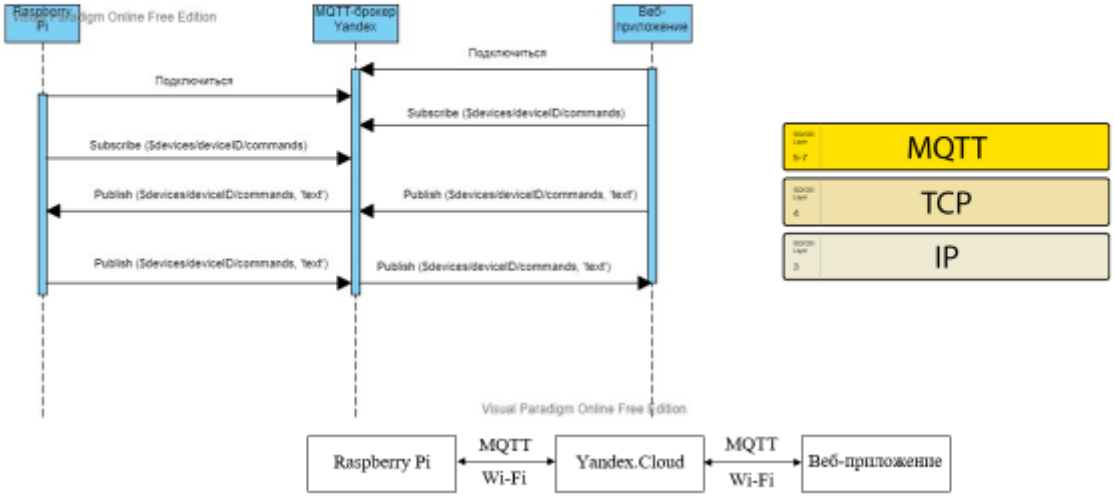
5

Выбор одноплатного компьютера

Наименование	Цена ($)	Процессор	ОЗУ	LAN	Wireless
NanoPi Neo Air	20	4x A7 @ 1.2ГГц	512Мб	Нет	WiFi, BT
Orange Pi PC 2	20	4x A53 @ 1.4ГГц	1Гб	Да	Нет
NanoPi2 Fire	23	4x A9 @ 1.4ГГц	1Гб	Да	Нет
MYS-6ULX SBC	25	1x Cortex-A7 @ 528MGz	256Мб	Да	WiFi
NanoPi A64	25	8x A53 @ 1.2ГГц	1Гб	Да	Нет
pcDuino Lite WiFi	29	1x A8 @ 1ГГц	512Мб	Да	Нет
NanoPi M2A / M3	30	4x A53 @ 1.4ГГц	1Гб	Да	WiFi, BT
Orange Pi Prime	30	4x A53 @ 1.4ГГц	2Гб	Да	WiFi, BT
Orange Pi Win Plus / Win	30	4x A53 @ 1.4ГГц	2Гб	Да	WiFi, BT
Odroid-C0	31	4x A5 @ 1.5ГГц	1Гб	Нет	Нет
NanoPi M1 Plus	35	4x A7 @ 1.2ГГц	1Гб	Да	WiFi, BT
Odroid-C1+	35	4x A5 @ 1.5ГГц	1Гб	Да	Нет
Raspberry Pi 2 Model B	35	4x A53 @ 900МГц	1Гб	Да	Нет
Raspberry Pi 3 Model B	35	4x A53 @ 1.2ГГц	1Гб	Да	WiFi, BT
B6Duino Zero	39	1 x86 @ 300МГц	128МБ	Да	Нет
Arduino Industrial 101	39	1x MIPS @ 400МГц	64Мб	Да	WiFi
BeagleBone Green	39	1x A8 @ 1ГГц	512Мб	Да	Нет

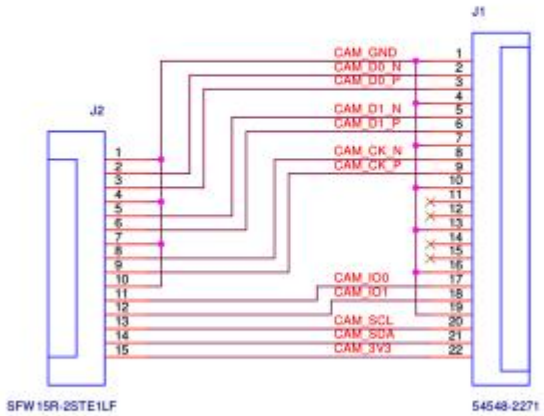
6

Взаимодействие Raspberry Pi с веб-приложением с помощью Yandex Cloud



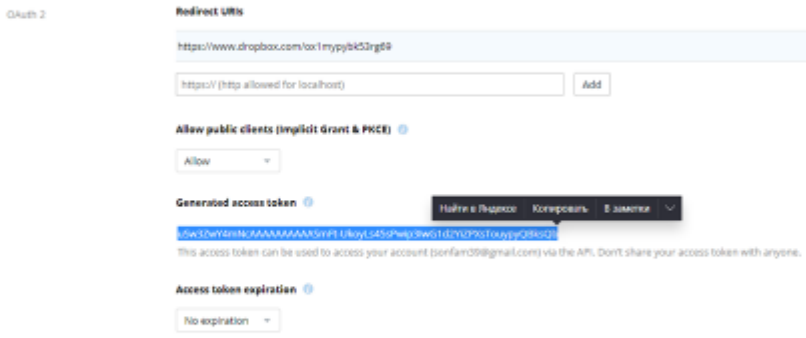
7

Взаимодействие Raspberry Pi с модулем камеры



8

Взаимодействие Raspberry Pi с файловым хостингом Dropbox

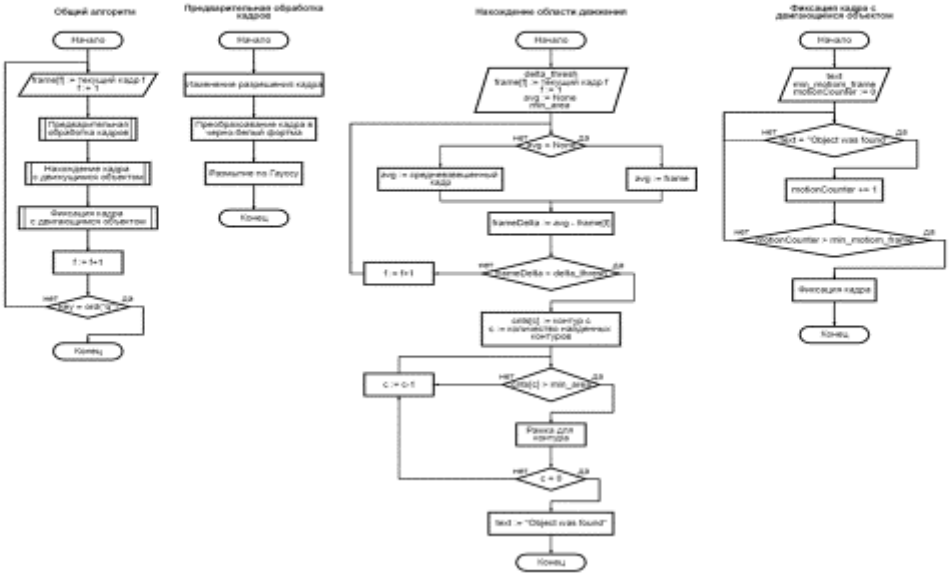



```
# check to see if the dropbox should be used
if conf['use_dropbox']:
    # connect to dropbox and start the session authorization process
    client = dropbox.Dropbox(conf['dropbox_access_token'])
    print('[SUCCESS] dropbox account linked')
```


```
{
    "show_video": true,
    "use_dropbox": true,
    "dropbox_access_token": "vpUhBKe4a3cAAAAAAAAAAbaoCOTgYdp8DuD5YmccPuoC3ZDc4y3dI3_mLKGDQ27u",
    "dropbox_base_path": "App/PI Image Search/Surveillance",
    "min_upload_seconds": 4.0,
    "min_motion_frames": 6,
    "camera_warmup_time": 2.5,
    "delta_thresh": 5,
    "resolution": [640, 480],
    "fps": 30,
    "min_area": 5000
}
```


9

Алгоритм обнаружения движения объекта



10

Алгоритм поиска регионов

11

Алгоритм распознавания объекта

12

Алгоритм работы Raspberry Pi с веб-приложением

13

Заключение

- Произведен анализ существующей системы распознавания, который показал, что она имеет ряд недостатков.
- Были сформированы требования к разрабатываемой программной части системы распознавания движущихся производственных объектов. Один из важнейших – возможность автономной работы при подключении к Web-приложению через облачную платформу Yandex Cloud.
- Были разработаны: функциональная модель распознавания движущихся объектов в автономном режиме, концептуальная модель взаимодействия web-приложения, облачной платформы, Raspberry Pi и модуля камеры.
- Были реализованы: основные алгоритмы системы распознавания движущихся производственных объектов, подключение Raspberry Pi к камере, подключение Raspberry Pi к файловому хостингу Dropbox, к Yandex.Cloud.

14